Defense rehearsal script

Daniel Blauvelt • Canonical defense • Updated November 12, 2026

This rehearsal script covers all 30 main slides and all 82 backup slides. The main talk is a continuous spoken script. Each substantive backup slide has a likely committee question, a self-contained spoken answer, and a deeper follow-up. The three navigation slides and the three reference pages have silent navigation cues. Read ordinary paragraphs aloud; headings, questions, and bracketed italic directions organize rehearsal and are silent.

The main talk contains approximately 6,000 spoken words: about 40 minutes at 150 words per minute and 48 minutes at 125, before pauses, animations, and questions. Backup responses are outside that timing. Rehearse the main talk aloud to establish its actual length.

Slide 9 states the three results. The footer highlights the current stage; backup footers show their deck page number. This script identifies each backup by its backup number and deck page. The clickable slide index and PDF bookmarks let you jump directly to a main slide or a backup response.

Slide 22 contains embedded animations with play controls. Playback has been checked in Firefox; desktop Acrobat Reader is also supported. Viewers without animation support show first-frame posters. The local aperture-demo.html remains an offline browser alternative.

Main talk index

Each entry links to its script page. Numbers identify audience-deck pages.

1 Title
2 Events alone recover a handheld office, a bullet hitting a cup, and Earth from orbit
3 Events alone recover a handheld office, a bullet hitting a cup, and Earth from orbit
4 Events record changes and the model supplies the brightness level
5 The route through the talk
6 Why an event camera? Each pixel reports its own changes, with microsecond timing and 140 dB range
7 Each event marks one threshold of log-brightness change, so the baseline level stays free
8 From events to images, motion, and depth: each estimate has its own unobserved quantity
9 Three estimators, each well posed
10 Event-derived models with explicit guarantees, built on established methods
11 Events fix each pixel's brightness increments and leave one constant free
12 Each kernel spreads an event's sign over nearby bins and conserves its total
13 Three discrepancies connect event sums to brightness changes
14 A unique, stable estimate for every positive weight
15 Positive weights curve the objective in every direction
16 One tridiagonal sweep per pixel, constant work per streamed frame
17 Temporal weights shrink as event activity grows
18 The penalty removes the baseline and slow drift but keeps passing edges
19 Events fix the scene structure and the penalty sets the baseline
20 Events alone recover scene structure with one convex solve and a unique global minimizer
21 Known motion lets every view constrain one scene
22 Three different motions look identical through a small window
23 One edge gives the normal speed, two orientations give the full motion
24 Three representations, one quadratic variational problem
25 Integrated directional information gives a unique flow in H^1
26 Two edge directions resolve more of the motion than one
27 Stereo matches temporal profiles by encouraging agreement
28 WLS scores the most points, 2.66M, at competitive depth error
29 Each next step starts from a result in the dissertation
30 What the data constrain and what the models prove

Backup responses index

Each entry links to its script page. Numbers identify audience-deck pages.

31 Discussion guide
32 Must every temporal penalty entry be positive?
33 The event measure has a jump primitive
34 Latent-signal gain and observation-noise gain are different
35 Kernel performance depends on the scene and adaptive weights
36 What the reconstruction scores measure
37 Can the reconstruction run in real time?
38 The weak equation supplies the natural boundary condition
39 Proof: control the mean and the fluctuation separately
40 Stability follows by subtracting weak equations
41 Discrete flow: connectivity and sampled directions matter
42 Why block-Jacobi converges for this graph model
43 Anisotropy needs a uniform ellipticity floor
44 A variable diffusion tensor must stay inside the divergence
45 The regression radius formula is a heuristic
46 Uniform and OLS weights agree through quadratics and differ on cubics
47 Mean-zero conditioning removes the unobserved mode
48 The selection theorem has an exact ellipsoid interpretation
49 An observation perturbation bound can protect a disparity winner
50 How the reciprocal IRLS contraction bound is obtained
51 The frozen-weight fixed point and the likelihood stationary point differ
52 What the implemented IRLS procedure returns
53 Depth evaluation distinguishes returned coverage and scored coverage
54 Exact zeros alone produce large pooled kurtosis
55 Adaptive temporal weights are fixed in the theorem
56 A real MVSEC flow example is qualitative
57 Quadratic H^1 regularization and TV answer different questions
58 Frozen weights match the value; an MM majorizer also matches the derivative
59 A Gaussian model gives an evidence interpretation of attenuation
60 The implemented estimator has several distinct layers
61 What does the dissertation contribute mathematically?
62 Literature anchors and where to find the claims
63 Stable estimates can still have regularization bias
64 The physical event stream loses more than one baseline
65 Integrated directional information is necessary for uniqueness
66 Coefficient stability is one stage of event-to-flow stability
67 Refinement requires a choice of continuum scaling
68 Strong smoothness selects one constant flow
69 Transporting a scalar differs from conserving a density
70 Correlated errors change the uncertainty; coercivity holds
71 Optical flow: further analysis
72 Time-surface gradient estimation: bias versus variance
73 Contrast maximization finds the motion that sharpens the events
74 How the known-motion comparison was evaluated
75 Contrast maximization changes the optimization problem
76 A global minimizer exists, even though the solver is local
77 Stereo depth: full Chapter 4 arc
78 Stereo depth in one slide: what Chapter 4 adds
79 Stereo alignment after motion compensation
80 At each candidate, couple two temporal profiles
81 Frozen weights give a unique, stable inner solve
82 Sum and difference modes expose the coupling
83 Profile out the latent variables to see what disparity compares
84 An attenuated match can cost less than the perfect one
85 Correct disparity needs a separation condition
86 A winning margin turns cost stability into selection stability
87 Depth sensitivity grows with distance
88 Heteroscedastic weights: statistical motivation and scope
89 IRLS: fixed-point existence and conditional convergence
90 Stereo benchmark: read accuracy alongside scored coverage
91 A weighting comparison supports the selected rule
92 Hard fusion changes both the constraint and the regularization
93 Motion-induced events and the aperture problem
94 One complementarity lemma behind the uniqueness results
95 Adaptive weights keep the temporal estimate locally Lipschitz
96 How the parameters were chosen
97 Learned estimators face the same free directions
98 The spline regularizer is uniformly coercive under refinement
99 Weak regularization: a limit in time, the aperture line in space
100 Where there are no events, smoothness fills in harmonically
101 A point measure needs a representation before optimization
102 Derive the time-surface constraint by following the edge
103 Voxel projection: transport a represented event-activity field
104 The structure tensor pools directional information
105 Proof idea: smoothness fixes variation, edges fix the drift
106 Stereo on MVSEC: competitive accuracy, more scored points
107 From event changes to well-posed estimators
108 How big is the temporal monocular problem?
109 One CPU core streams about 114 million pixel updates per second, with 16 bytes of state per pixel
110 References (1 of 3)
111 References (2 of 3)
112 References (3 of 3)

SLIDE 1

Title

Thank you, Professor Antil, for your guidance throughout this work, and thank you, Professors Griva, Warma, and Löhner, for serving on my committee and for your consideration and feedback on this work.

My dissertation develops variational and optimization methods for event cameras, sensors that report only changes in brightness.

Let me start with what these methods produce.

[Advance.]

SLIDE 2

Events alone recover a handheld office, a bullet hitting a cup, and Earth from orbit

[Pause so the committee can look at all three panels.]

These three images were reconstructed from event-camera data alone, with no ordinary frames, by the method in my second chapter.

On the left is a handheld recording in an office; the desk, the chair, and the person are recognizable. In the middle, a bullet strikes a ceramic cup. The events resolve the fracture and the spray of fragments, which an ordinary camera at a standard frame rate would blur. On the right is the Earth seen from the International Space Station by the Falcon Neuro event cameras: the Honduran coastline and clouds, from a slowly changing scene with sparse events.

[Advance to the false-color version.]

SLIDE 3

Events alone recover a handheld office, a bullet hitting a cup, and Earth from orbit

These are the same three reconstructions, now in false color; color and grayscale are display choices for the same reconstructed brightness.

Why does this matter? An event camera reports each pixel's brightness changes with microsecond timing and low power. That suits fast scenes, difficult lighting, and platforms where power and computation are scarce, such as drones and spacecraft like this one. On those platforms the estimator also has to be cheap and predictable.

So the dissertation asks: from changes alone, what can we recover, with what guarantees, and at what cost? I'll answer for images, motion, and depth.

[If asked whether the reconstruction runs in real time: slide 16, Backup 79, page 109, and Backup 07, page 37.]

SLIDE 4

Events record changes and the model supplies the brightness level

[Point to the reference image, the accumulated events, and the reconstruction.]

On the left is an ordinary camera image of an outdoor scene; we keep it there to help recognize the scene. In the middle is a fifty-millisecond sample of the events from the same recording. Orange marks pixels that got brighter, blue pixels that got darker, and purple pixels that did both. On the right is our reconstruction, computed from a longer stretch of those events, a little under a second.

Each event says only that the brightness at one pixel changed at one moment. So the events record change; the brightness level comes from the model. To get from events to the image on the right, the model has to fill in what the events leave out, starting with a reference brightness level. Regularization makes that choice and balances it against fitting the data.

That is the theme of the talk. At each step, I'll separate what the observations determine from what the model chooses. Regularization can pick out a single answer. To know what that answer means, we need to know which parts came from the data and which came from the model.

Here's how the talk is organized.

[If asked whether the baseline is all an event camera loses: Backup 34, page 64.]

SLIDE 5

The route through the talk

[Read down the five rows.]

Here's the plan. First, what an event camera reports, and the three problems that come out of it. Then the largest part, reconstructing an image over time from a single pixel's changes: the model, why its estimate is unique and stable, what it costs, and how it does on real data. Then optical flow: why motion is only partly visible through a small window, and when edges pin it down. A short summary of stereo depth, and then next steps. The footer tracks where we are.

Start with the sensor.

SLIDE 6

Why an event camera? Each pixel reports its own changes, with microsecond timing and 140 dB range

[Trace the moving bar through the two diagrams.]

A conventional camera samples the whole image at fixed frame times. In the left diagram, each vertical column is one frame across all the pixels. The dashed diagonal line is an edge moving across the scene the pixels are viewing: time runs left to right, and the pixels are stacked vertically.

In an event camera, each pixel works on its own. When the leading edge of this bright bar reaches a pixel, the pixel gets brighter and fires a positive event. When the trailing edge passes, it gets darker and fires a negative event.

The output is a stream of locations, timestamps, and signs. That's why event displays tend to look like moving outlines. Under steady lighting, a pixel's brightness changes when a boundary crosses it. The inside of a uniform region produces little activity even while it moves, and so does a straight edge sliding along itself, because the local pattern stays the same. Changes in lighting trigger events too, since the sensor responds to any change in brightness over time.

Event sensors also have microsecond timing and high dynamic range, so they suit fast motion and difficult lighting.

For the mathematics, the key feature is that each pixel measures change relative to its own reference. Let me make that rule precise.

SLIDE 7

Each event marks one threshold of log-brightness change, so the baseline level stays free

[Follow the solid curve and point to one positive and one negative event.]

The solid curve is the log intensity at one pixel. The dashed curve is the reference the pixel compares against.

When the log intensity rises one threshold above that reference, the pixel reports a positive event and resets its reference. A drop of one threshold gives a negative event. Every pixel resets on its own schedule, which is why event times are asynchronous.

The equation on the right states this ideal rule. An event carries a pixel location, a time, and a polarity, and the change since the last reset is the polarity times the threshold.

Now shift the whole log-intensity curve up by a constant. Every change stays the same, so the events stay the same. That's the first ambiguity in the talk: the events leave the baseline level free.

Real sensors add threshold variation and other errors. We use the ideal rule to motivate the observation model, and then pose reconstruction as a regularized fit to the data.

[If asked about the continuous-time version of the event rule: Backup 03, page 33. If asked whether the baseline is all the sensor loses: Backup 34, page 64.]

SLIDE 8

From events to images, motion, and depth: each estimate has its own unobserved quantity

[Indicate the three panels from left to right.]

These three estimates come from MVSEC, the Multi-Vehicle Stereo Event Camera dataset. It records a synchronized pair of event cameras on a drone flying indoors and on a car driving outdoors. The cameras also capture ordinary frames, which we use as reference images, and the dataset includes lidar-derived ground-truth depth, which we'll use for stereo. All three estimates here come from one indoor flight, at nearly the same moment. Each method uses its own observation window, and the depth estimate uses two cameras.

On the left is an image reconstructed by the temporal model. In the middle is optical flow, the apparent motion in the image plane. Color shows motion direction and magnitude over the observation window; white marks pixels with no events. For example, the yellow-green floor at the bottom is moving down the image. On the right is depth from the two event cameras, with the color scale running from half a meter to eight meters.

Each problem has its own difficulty. For reconstruction, the brightness baseline is unknown. For flow, a local edge reveals the motion across it and leaves the motion along it free. For stereo, we have to decide which points in the two views are the same point in the scene.

Here's what the talk shows for each of the three.

[If asked which methods produced these panels: a Gaussian temporal kernel (sigma 5 ms) with sigmoid weights, the selected Chapter 2 configuration; contrast maximization, and weighted least squares. Contrast maximization has a nonconvex objective outside the quadratic flow family analyzed later; its existence result is Backup 46, page 76. If asked about the spline-parameterized flow regularizer under refinement: Backup 68, page 98. If asked what the stereo method is, in one slide: Backup 48, page 78. If asked about the quality of MVSEC's ground truth: Backup 60, page 90, for depth; Backup 26, page 56, for flow.]

SLIDE 9

Three estimators, each well posed

[Move down each column.]

Each column answers the open quantity from the last slide.

For reconstruction, the penalty supplies the brightness baseline. Each pixel's history comes from one tridiagonal linear solve, and the estimate is unique and stable for every choice of positive weights. That's the largest part of the talk, including what it costs to compute.

For optical flow, smoothness supplies the motion along each edge. Three different ways of turning events into motion constraints fit into one quadratic variational problem, and the flow is unique as soon as the scene has edges in two different directions.

For stereo depth, the model encourages agreement between the two views' temporal profiles and compares candidate matches. With the weights frozen, the inner problem at each candidate depth has a unique solution, and on the MVSEC benchmark the method is competitive in accuracy and scores the most points.

The common question behind all three: which quantities do the events determine, and which does the model supply? First, where this work sits among existing methods.

[If asked for the literature behind each model: Backup 32, page 62. If asked what is mathematically new: Backup 31, page 61.]

SLIDE 10

Event-derived models with explicit guarantees, built on established methods

[Compare the prior approaches with the dissertation contribution in each column.]

For image reconstruction, Rebecq and colleagues train a recurrent neural network on simulated events. Zhang, Yezzi, and Gallego use optical flow to pose reconstruction as a linear inverse problem, with classical or learned regularization. Antil and Sayre combine events with conventional frames in a bilevel problem that estimates a parameter of the event-based blur kernel.

Our published temporal work reconstructs each pixel on its own from event increments. I develop the temporal kernel representation and the activity-based penalties, and then analyze the resulting quadratic solve: what the observations determine, how regularization picks the baseline, and how perturbations move the solution. That analysis shows where the reconstructed temporal structure comes from.

For flow, Horn and Schunck gave us the data-plus-smoothness formulation, and Benosman and colleagues developed the local geometry of event timestamps. Classical quadratic flow already has a well-posedness theory, for example from Weickert and Schnörr. What I add is one quadratic family that covers three ways of building coefficients from events, with explicit assumptions for when the motion is identified and how sensitive the estimate is to those coefficients.

Now the reconstruction model, starting with a single pixel.

[If asked what is mathematically new when the tools are classical: Backup 31, page 61; the mechanism shared by all three chapters is Backup 64, page 94. If asked for the literature behind each model: Backup 32, page 62. If asked why not train a neural network: Backup 67, page 97.]

SLIDE 11

Events fix each pixel's brightness increments and leave one constant free

Here's the temporal model in scene terms. The unknown is each pixel's log-brightness history over the recording. The events tell us how much that brightness rose or fell in each interval. Each pixel is reconstructed on its own, and the model picks the reference level that the events leave open. A pixel with no events stays at that reference level, however bright it really is.

Now the details. Fix a pixel and take its events over a time window. At each event time, place a kernel and multiply it by the polarity. Adding these up gives the represented temporal signal on the left.

Then we integrate over the reconstruction intervals. Each entry of y is the signed event contribution in one interval. The algorithm chooses the reconstruction times.

[Point to the first row of A.]

The matrix A takes differences of consecutive samples of the unknown profile. Its first row subtracts the first sample from the second, and each later row does the same one step further on. So A f should approximately equal y. These are plain increments; dividing by the time step would turn them into difference quotients.

If A f is zero, every consecutive pair is equal, so f is constant. And every constant vector gives zero differences. So the null space is exactly the constants.

In short, the data constrain the increments, and we need one more rule to choose the level.

[If asked whether the constant is all the information the sensor loses: Backup 34, page 64. If asked which choices fix the level: Backup 02, page 32; for a mean-zero formulation: Backup 17, page 47. If asked what space the model lives in: the theorems are in R^N, one coordinate per reconstruction time, with a weighted l two norm. The continuous-time version is posed in H one for a regular kernel, and in the space of bounded-variation functions in the delta limit, Backup 03, page 33.]

SLIDE 12

Each kernel spreads an event's sign over nearby bins and conserves its total

[Point to the kernel curves, then to the bin-integral formula.]

The kernel controls how an event's contribution spreads over time. Delta binning puts the whole event into its own bin. The rectangular, spline, and truncated Gaussian kernels spread it over a short interval.

Delta binning is natural when we trust event timing at our reconstruction scale. Gaussian weighting is motivated by uncertainty concentrated near the timestamp, while rectangular weighting treats a bounded neighborhood equally. Splines provide local averaging with controlled smoothness. These interpretations motivate the alternatives; our experiments analyze which trade-off works better for the evaluated scenes.

Each kernel has unit mass, so an event contributes its full signed amount whenever its whole support lies inside the observation window. The sum on the right expresses that conservation across the bins. Events whose support crosses the edge of the window need special handling at the boundary.

A wider kernel spreads the contribution further. That can reduce sensitivity to timing jitter, and it also blurs fast changes. The curves here share one width for illustration; the experiments use different widths for different kernels.

The difference matrix A stays the same. The kernel changes y, and through y it can also change the weights. That will matter when we get to conditioning.

Next: when does this integrated observation match a physical brightness change?

[If asked why the kernel changes conditioning when A stays fixed, or which kernel is best: Backup 05, page 35.]

SLIDE 13

Three discrepancies connect event sums to brightness changes

So far, y is something we compute from the events. This identity from Chapter 2 says exactly how it relates to physical brightness change.

[Point to the identity.]

Between any two times a and b at one pixel, the change in log brightness, in threshold units, equals the event sum y plus three corrections. Each panel below shows one of them.

[Point to the panels from left to right.]

First, kernel spill. The event occurs inside the bin, but part of its kernel lies beyond the right boundary. The shaded mass is excluded from this bin's integral. For this positive event, B is negative, so subtracting B restores its contribution to the signed count.

Second, overshoot. The dashed level is one threshold above the last reset. The recorded event lies above that level. The marked excess is epsilon divided by the threshold, because the vertical axis is normalized log brightness.

Third, sub-threshold change. Brightness changes after the last event and stays below the next threshold. The vertical gap at b is r of b. There is a corresponding gap at a, so the identity uses their difference.

Kernel spill comes from the representation and bin boundaries; overshoot and the sub-threshold change connect the events to physical brightness. When the corrections are small, A f approximately equal to y models brightness changes. The absolute level comes from the model. Next is how the model chooses it.

[If asked about missed or spurious events: they can make epsilon comparable to the threshold. For the continuous-time limit: Backup 03, the jump primitive. For signals that stay below threshold throughout a window: Backup 34.]

SLIDE 14

A unique, stable estimate for every positive weight

The objective has two terms. The first fits the reconstructed increments to y. The second penalizes the size of the profile, weighted by a positive diagonal matrix, Lambda, held fixed for the theorem.

[Point to the three boxes from left to right.]

The theorem gives three properties. First, uniqueness. Setting the gradient to zero gives the normal equations, with matrix K time, A transpose A plus Lambda squared. That matrix is bounded below by the smallest weight squared, so it is invertible, and the unique minimizer is its inverse applied to A transpose y.

Second, stability. One spurious or missed event moves a pixel's history by at most two over the smallest weight squared.

Third, the reference level. Take the inner product of the normal equations with the constant vector. A sends constants to zero, so the weighted average of the solution is zero. In scene terms, the quiet intervals carry the largest weights, and they set each pixel's reference level. The penalty also adjusts the fitted increments, because the solution balances both terms.

Here's why the first two hold.

[If asked whether every weight must be positive: Backup 02, page 32. If asked how lambda should scale when the time grid is refined: Backup 37, page 67. If asked about a mean-zero formulation: Backup 17, page 47. If asked how close the estimate is to the true signal: Backup 33, page 63. If asked about recomputed weights: Backup 65, page 95.]

SLIDE 15

Positive weights curve the objective in every direction

[Walk through the first inequality slowly.]

Take any nonzero direction z. The quadratic form of the normal matrix is the squared norm of A z plus the squared norm of Lambda z.

The first term is nonnegative. The second is at least the smallest weight squared times the squared norm of z. Since that weight is positive, the curvature is positive in every direction.

That gives coercivity and strict convexity, so a minimizer exists and is unique.

For stability, compare two data vectors with the same weights. Subtracting their normal equations gives the exact difference between the two solutions, shown here. Bounding the inverse matrix gives the estimate. In scene terms, one spurious or missed event changes the data by at most one in norm, because each kernel splits a single unit of polarity across the bins. So the bound limits how far one noise event can move a pixel's history, and the smallest weight sets that limit.

The last expression bounds the condition number in terms of the largest and smallest weights. Lowering the smallest weight weakens these worst-case guarantees.

All of this holds for fixed weights. If the data change and we also recompute the weights, the comparison picks up an extra term for the change in the weights.

The same matrix also makes the solve cheap.

[If asked what happens as the weights go to zero: Backup 69, page 99. If asked about recomputed weights: Backup 65, page 95. If asked how close the stable estimate is to the true signal: Backup 33, page 63. If asked about conditioning as lambda becomes small: Backup 17, page 47. If asked how the penalty treats signal and noise differently: Backup 04, page 34.]

SLIDE 16

One tridiagonal sweep per pixel, constant work per streamed frame

At the pre-defense, the committee asked what this costs, and whether it could run on small, low-power hardware. The structure of the matrix answers the cost question.

[Point to the matrix.]

A takes consecutive differences, so A transpose A has twos on the diagonal, ones at the ends, and minus ones beside the diagonal. Adding the squared positive weights preserves that structure and makes each diagonal entry exceed its off-diagonal sum by lambda i squared, so the sweep needs no pivoting. One forward sweep and one backward sweep solve each pixel's system in time proportional to the number of samples. Across the image, that is O(H W N_t). A pixel whose whole observation vector is zero has the exact zero solution and can skip the solve.

If only the newest frame is needed, the forward recurrence returns the endpoint of the batch solution on the history accumulated so far. It keeps three recurrence numbers plus an output value per pixel. Each new interval costs constant work per pixel; older frames would need backward substitution to be revised.

[Point to the bottom line.]

On an MVSEC recording, the tridiagonal solve takes three hundredths of a second where the dense solve takes one and a half, and it agrees with the MATLAB reference to rounding error.

On one CPU core, the streaming form keeps up with a 346 by 260 sensor producing a frame every two milliseconds.

Next, how the weights are chosen.

[For per-core timings and the pixel budget: Backup 79, page 109. For problem size and low-power strategies: Backup 78, page 108.]

SLIDE 17

Temporal weights shrink as event activity grows

Lambda controls how strongly the reconstruction is pulled toward its reference level, and all three rules reduce that pull as the magnitude of net signed activity increases. Sigmoid gives a smooth, bounded transition; inverse-proportional weighting retains a longer tail of shrinkage; and exponential weighting drops rapidly until its input clamp is reached. At our selected scales, inverse-proportional weighting applies the strongest shrinkage.

The rule we selected uses the size of the signed event total in each bin. Positive and negative events can cancel in that total, so it measures net signed activity.

[Point to the sigmoid expression.]

As activity increases, the sigmoid rule lowers the penalty. A positive floor keeps every weight above zero. So quiet intervals get the largest weights, and through the weighted-average identity they set each pixel's reference level. Busy intervals get the smallest weights: they're free to change, and they set the worst-case noise bound from the stability result.

We chose each rule's scale by grid sweep, and with the Gaussian kernel the sigmoid rule gives the best average rank over the nine evaluation metrics. Among the three rules it gives the smoothest results in time, at a higher condition number: about 265, against 2.5 for inverse-proportional and 18 for exponential weighting outdoors. A condition number of 265 uses about two and a half of the sixteen significant digits of double precision.

The point to remember is that the weights are computed from the data and then held fixed during the solve. The theorem covers that frozen problem. Stability of the full adaptive procedure also requires controlling how the weights change.

Next, what the penalty does to the reconstructed signal.

[If asked how data-computed weights fit the fixed-weight theorem: Backup 25, page 55; for the full adaptive map: Backup 65, page 95. If asked how the scales were chosen, or about the discrepancy principle: Backup 66, page 96. If asked how kernel and weight choices change conditioning: Backup 05, page 35.]

SLIDE 18

The penalty removes the baseline and slow drift but keeps passing edges

To interpret the penalty, take a single scalar weight, so Lambda is lambda times the identity. And suppose the observations are exact differences of a latent signal.

The eigenvectors of A transpose A are temporal modes. In that basis, reconstruction multiplies each mode of the latent signal by g over g plus lambda squared, where g is the mode's eigenvalue.

[Point first to zero frequency, then along the two curves.]

The constant mode has eigenvalue zero, so reconstruction removes it entirely. Among the remaining modes, the slowest are damped the most. In the images, slow brightness drift fades and passing edges remain, which is why the reconstructions look edge-enhanced. A larger lambda damps more.

That's what the penalty does to the latent signal. Kernel averaging is a separate step, and it can blur fast changes before the solve.

The curves connect the finite-grid gains so the trend is visible. A general positive diagonal still gives the uniqueness theorem, but its effect on the modes needs a fuller matrix analysis.

Next, what these reconstructions look like on real data.

[If asked what the penalty does to measurement noise: Backup 04, page 34. If asked about bias in the damped modes: Backup 33, page 63. If asked what happens as lambda goes to zero: Backup 69, page 99.]

SLIDE 19

Events fix the scene structure and the penalty sets the baseline

[Point to the images, then to the table.]

The pictures are a representative example from MVSEC indoor flying two, the dataset from slide 8: a reference image from the ordinary frames, and an event-only reconstruction with the selected configuration: a five-millisecond Gaussian kernel and sigmoid weights. The table summarizes the kernel comparison over the indoor flying sequences.

The reconstruction makes the scene structure visible. The table reports two things: PSNR, which measures similarity to the reference image, and the condition number of the linear system. PSNR also scores the absolute brightness level, which the model selects, so it measures something different from agreement with the recorded changes.

Indoors, the spline and Gaussian kernels give the highest PSNR, about 0.2 decibels above delta binning, and all three kernels give condition numbers near 260. Outdoors, the PSNR ordering changes. So the kernel moves reconstruction quality a little, and the weight rule sets the conditioning.

The kernel changes the observations, and the adaptive weights can then change the matrix we solve. That's how representation choices reach the linear system.

Next, how these reconstructions compare with the bilevel approach on MVSEC.

[If asked why the PSNR values are low, or what the metrics measure: Backup 06, page 36. If asked about the outdoor ranking or the kernel choice: Backup 05, page 35.]

SLIDE 20

Events alone recover scene structure with one convex solve and a unique global minimizer

[Point to the three images, then down the columns.]

This is one outdoor frame from MVSEC. First the camera's reference frame. Then the two outputs the bilevel method defines: its reconstruction, which starts from the camera frame and corrects the blur, so it closely matches the frame it was fitted to; and its dynamic part, the component the events drive. Last, the event-only reconstruction, on the same display scale as the dynamic part. Both recover the tree, the parked vans, and the glare off the hood; the dynamic part also carries texture from the frames. The overall brightness of the event-only image is the level the model selected, since the events constrain only changes.

Below the images is the relation to the Antil–Sayre bilevel approach. In this chapter, the events set activity-based weights, and then we do one fixed convex solve over time. The bilevel approach uses events together with intensity frames, and its outer optimization adjusts z, a parameter of the event-based blur kernel that links each blurry frame to the sharp latent image. So their outer variable shapes how events explain the frames, while my rule chooses the magnitude weights of the temporal profile.

The table compares complete methods with different inputs. The bilevel PSNR, about 40 dB indoors and 37 outdoors, is measured on the frames it is fitted to, so it shows how closely its reconstruction reproduces its own input frames; the event-only reconstruction scores 8.0 and 7.2. PSNR compares with a reference frame, so it also scores the absolute brightness, which the events leave to the model.

The last row compares guarantees. The bilevel analysis proves existence, and local uniqueness under additional conditions; the result I presented gives a unique global minimizer for every fixed positive diagonal.

Last in this chapter: what changes when the pixels share one scene.

[If asked how the bilevel scores on frames it was not fitted to: 24.3 dB indoors and 25.2 outdoors, the same as interpolating between the camera frames. If asked about the outdoor kernel ranking: Backup 05, page 35. If asked how PSNR and condition number are computed: Backup 06, page 36. If asked which prior work this builds on: Backup 32, page 62.]

SLIDE 21

Known motion lets every view constrain one scene

On the International Space Station, motion is the enemy of a frame camera. For an event camera with a known orbit, motion is what's measured.

The station moves at about seven point seven kilometers per second, so any exposure long enough to gather light smears the ground. An event pixel timestamps each change to the microsecond, covers about 140 decibels, and sends only changes, which suits a payload with little power. And because the orbit is known, every view sees the same ground through a known motion.

In joint work with Professor Antil and Dr. Sayre, called ESAR, we use exactly that: the same event constraints, now describing one static scene seen by a moving camera.

[Top lane: the operator equation.]

Theta is the scene. P holds the known motion: it maps the scene into each view. A takes differences between consecutive views, as before. The name borrows from synthetic aperture radar, which combines a moving sensor's views into one image.

We fit the scene to all the event counts at once: least squares plus a penalty, as for a single pixel. With a quadratic penalty, it's one linear system, and the dissertation proves the solution is unique and bounds how far noise can move it.

[Middle lane: the simulated disks.]

The simulated disks show the mathematics. The error concentrates along rows through the disks, because a pattern that is constant along the motion looks the same in every view. It's the same lesson as the baseline: find what the observations control, and let regularization supply the rest.

[Bottom lane: the orbital pair.]

This is a real Falcon Neuro pass looking straight down. On the left, each pixel is reconstructed on its own, and the structure is faint and noisy. On the right, every view constrains one shared scene, and the coastline and the land texture come out. The motion that blurs a frame camera gives the event camera more views of the same ground.

Here the motion was known. In optical flow, the motion is the unknown.

[If asked what regularization supplies when the observations leave directions free: Backup 64, page 94. If asked about related prior work: Backup 32, page 62.]

SLIDE 22

Three different motions look identical through a small window

A drone that stabilizes itself, dodges an obstacle, or tracks its own motion needs to know how the image is moving, right now. A frame camera infers motion by comparing two pictures. At thirty or sixty frames a second, a fast edge jumps many pixels between them and blurs within them, and the answer arrives a frame late. An event camera timestamps the edge as it crosses each pixel, to the microsecond. Those crossing times are the motion measurement: their slope gives the edge's speed. The catch is that a single edge reveals only part of its motion.

Optical flow is the apparent velocity in the image plane. A local view can leave part of that velocity free. This is the aperture problem; an aperture here just means a limited window.

[Play the embedded animation in the left panel.]

Look at the stripes through these three circular windows. They all seem to move the same way. Each window shows only the inside of a straight striped pattern, so all we can see is how fast the stripes move across their own edges.

[Play the embedded animation in the right panel. Pause.]

With the full patterns, we see three different motions: diagonal, upward, and leftward. All three share the same motion perpendicular to the stripes. They differ in the motion along the stripes, and the window hides that part.

The animations show the geometry, and the same ambiguity holds for an event camera. Under steady lighting, these ideal patterns produce the same brightness changes inside each window, and so the same events, given the same thresholds and starting references.

Next, I'll draw this ambiguity for a single edge and state what the flow model assumes.

[If asked to derive the aperture problem from the event-generation model: Backup 63, page 93. If asked when the data do determine the motion: Backup 35, page 65.]

SLIDE 23

One edge gives the normal speed, two orientations give the full motion

[Point to the left diagram: the edge, the teal arrow, then the amber arrow.]

First, what the camera sees. An edge in the scene is where the light changes sharply: a dark object's outline against a bright wall, or a painted stripe. As this edge sweeps over a pixel, the pixel fires, so the firing times trace where the edge is.

The teal arrow is the edge's motion perpendicular to itself; the dashed line is where it will be a moment later. The amber arrow is motion along the edge. Sliding a straight edge along itself, with its ends out of view, leaves every pixel's brightness unchanged, so the events fix the teal part and leave the amber part free.

[Point to the right diagram.]

The right panel plots possible velocities. Each line is every velocity that fits one edge's events. Edge 1 is vertical, so its events fix the horizontal speed, the teal arrow, and leave the vertical speed free.

A second edge, elsewhere in the image and oriented differently, gives the diagonal line. If both edges move with the same velocity, the crossing picks it out: the ringed dot, v. It's the teal part edge 1 measured, plus the amber part edge 2 fixed.

The two edges can be anywhere in the window. The model assumes neighboring pixels move alike, and that carries each edge's information across the image. The crossing is the true velocity when the edges really move together: two sides of one object translating across the view, or a camera pan. That's a modeling assumption. Non-parallel lines always cross, so if the edges belong to objects moving differently, the model still returns a velocity, and it's wrong for both.

So the flow model is: the unknown is the image velocity at every pixel; events give each edge's perpendicular speed; neighbors moving alike fill in the rest. The three representations differ in how they get the edge measurement: crossing times, transported event activity, or gradients pooled over a small window.

The condition to remember is edges in two different orientations, anywhere in the recording: their constraint lines cross, and the crossing fixes the velocity. On slide 25 it decides whether the answer is unique. Next, the three event representations that supply each edge's measurement.

[If asked whether two orientations anywhere suffice: slide 25; necessity, Backup 35, page 65. If asked what links distant edges: with exact data and one shared velocity, the constant field fits every edge and has no gradient, so it is the unique minimizer at any alpha and separation. If objects move differently but each has a corner or texture: the events contradict one velocity, and smoothing blends the motions across the boundary, Backup 27, page 57, and Backup 70, page 100.]

SLIDE 24

Three representations, one quadratic variational problem

[Point to the energy, then briefly indicate the three table rows.]

This keeps the classical data-plus-smoothness structure, with the event representation supplying the coefficients.

The first integral measures agreement with the motion constraints. B is the data matrix: it records which motion directions the local edges measure, and how strongly. The forcing b carries the measured values. The second integral penalizes spatial variation of the velocity, with positive strength alpha squared.

For the time surface, B is the outer product of the time-surface gradient, from following an edge's crossing times. For voxel projection, it's the outer product of the activity gradient, from transporting event activity with the scene. For the structure tensor, it's the pooled spatial block, from gradients over a small neighborhood.

The natural space is H one on the image domain, for both velocity components: square-integrable velocities with square-integrable weak derivatives.

The advantage is that we prove one result for a fixed data matrix B and forcing b, and then apply it to each representation once we've checked its hypotheses.

The question now is whether the data and the smoothness term, together, control every direction in that space.

[If asked for the derivations: time surface, Backup 72, page 102; voxel projection, Backup 73, page 103; structure tensor, Backup 74, page 104. If asked why H one rather than total variation: Backup 27, page 57. If asked which boundary condition the model uses: Backup 08, page 38. If asked about direction-dependent smoothing: Backup 13, page 43, and Backup 14, page 44. If asked how the model is solved on a grid: Backup 11, page 41, and Backup 12, page 42.]

SLIDE 25

Integrated directional information gives a unique flow in H^1

This result specializes the classical well-posedness framework to the event-derived coefficients, under these hypotheses.

[Explain each hypothesis by its role.]

The domain is bounded, connected, and Lipschitz, and the smoothness weight is positive. The data matrix B is bounded, symmetric, and positive semidefinite, and the forcing b is square-integrable.

The key assumption is that the integral of B over the domain is positive definite. Under these hypotheses, the energy has a unique minimizer in H one.

What does that integrated condition mean? Take any nonzero constant velocity. Summed over the domain, the observations must give positive energy in that direction. In scene terms, the recording has edges in at least two orientations, possibly at different pixels: the two crossing lines from the velocity diagram.

This is the requirement from slide 23: the one hypothesis the scene has to supply, while the others come from how we set up the problem. If every edge is parallel, as inside a stripe pattern, a uniform slide along the stripes costs nothing, and many answers fit equally well. The theorem promises one answer; whether it's the true motion depends on slide 23's assumption that neighbors move alike, and slide 26 measures how close it comes.

Individual locations may give only one direction. Regions with no texture or no events give zero data curvature, and their neighbors supply the motion there.

Connectedness matters because it makes any field with zero gradient a single constant vector over the whole domain. The integrated matrix controls that remaining freedom.

The proof follows the picture. Split the velocity into its mean and a zero-mean fluctuation. Smoothness controls the fluctuation, by Poincaré's inequality; the integrated data matrix controls the mean, the uniform drift that smoothness can't see. Together they give coercivity, and Lax–Milgram gives the unique minimizer. Next, two examples where we know the true motion.

[If asked for the proof: the idea is Backup 75, page 105; the full proof is Backup 09, page 39. If asked what the estimate does where there are no events: it is the harmonic fill-in of the surrounding motion, Backup 70, page 100. If asked how to check the condition on data: the discrete version is the smallest eigenvalue of the two-by-two sum of sampled data blocks, Backup 11, page 41; its size is the margin m. If asked about a disconnected domain: each component carries its own constant drift and needs its own integrated condition. If asked whether the integrated condition is necessary: Backup 35, page 65. If asked whether the convergence proof covers the solvers used in the experiments: Backup 12, page 42. If asked what happens as alpha goes to zero or grows large: Backup 69, page 99, and Backup 38, page 68.]

SLIDE 26

Two edge directions resolve more of the motion than one

[Give the audience a moment to compare the two rows.]

These selected simulation windows illustrate the directional-information argument, using a known diagonal motion. The top row has a stripe texture, and the bottom row a checkerboard. The true-motion arrows and the estimates share a common scale.

Inside a vertical stripe region, the strong gradients are horizontal, so they mainly constrain the normal motion. In the theorem's terms, this texture gives the integrated matrix little energy along the stripes. In the velocity diagram from earlier, the stripe interior gives only one line. The checkerboard has edges along both axes, so its lines cross, which reduces the aperture ambiguity. Object boundaries can add directional information as well. How accurate the estimate is also depends on the representation and the regularization.

Last, a short summary of stereo depth.

[If asked about other methods: the contrast-maximization comparison on these scenes is Backup 43; "why analyze the quadratic family" is Backup 45. If asked how the examples were selected and scored: Backup 44, page 74. If asked about real-data flow: Backup 26, page 56. If asked whether contrast maximization has a global minimizer: Backup 46, page 76. If asked whether the spline regularizer loses coercivity under refinement: Backup 68, page 98.]

SLIDE 27

Stereo matches temporal profiles by encouraging agreement

I'll close the technical part with stereo depth.

A vehicle needs metric depth: how far away the obstacle is. One camera leaves the scale free; two cameras fix it. Frame stereo matches brightness patterns, and blur, glare, and darkness corrupt those patterns. Two event cameras watching the same edge see it change at the same moment, so time becomes the matching signal.

[Point to the diagram.]

Two calibrated cameras see the same scene point. After rectification the image rows line up, so matching is a search along one row, and the horizontal offset is the disparity. By similar triangles, depth is the focal length times the baseline, divided by the disparity.

[Point to the objective.]

For each pixel and each candidate disparity, the objective at the top fits a temporal profile to each view: the first group is the Chapter 2 problem for each camera, and the last term, weighted by gamma, ties the two profiles together softly. The cost scores how well one shared explanation fits both. We call it WLS, for weighted least squares. The weights treat busy intervals as noisier, following a Poisson count model; they're updated between solves and frozen within each one. For a fixed candidate with frozen weights, the inner problem is the temporal argument with one more nonnegative term, so its solution is unique. Choosing the disparity is a separate comparison across candidates, after semi-global matching aggregates the costs.

The challenge is that only edges fire, so the hard part is how much of the scene gets a depth at all. That is where WLS leads.

[If asked why the inner solve is unique: Backup 51, page 81. If asked whether a unique solve gives the right disparity: Backup 54, page 84; the conditions that do are Backup 55, page 85, and Backup 56, page 86. If asked whether IRLS converges: Backup 59, page 89. If asked why the weights treat busy intervals as noisier: Backup 58, page 88. For the full stereo argument: Backup 47, page 77; in one slide, Backup 48, page 78.]

SLIDE 28

WLS scores the most points, 2.66M, at competitive depth error

[Point to the table, then the bars.]

On the MVSEC indoor flights, against lidar-based ground truth, WLS has the lowest error on flight two and the lowest log RMSE. The adaptation of Piatkowska and colleagues' method is lower on the other three flights, by one and a half to six centimeters. WLS scores two point six six million points, the most of the five methods and more than twice Piatkowska's. Each method is scored on its own valid pixels, so read accuracy and coverage together.

[If asked for the full table: Backup 76, page 106. If asked whether the comparison is fair: Backup 60, page 90.]

SLIDE 29

Each next step starts from a result in the dissertation

[Move through the four boxes, left to right.]

First, event-sensor modeling. Slide ten separated the measured increments from brightness changes by three error terms. Measuring those terms on a real sensor, together with timestamp and binning error, would let the stability bounds translate sensor error into estimate error. The GMU Neuromorphic Imaging and Digital Twins Lab makes those controlled acquisitions possible.

Second, learned temporal regularization, as a possible extension. When all the weights are positive, the fixed-weight solution is a smooth function of those weights. That gives a well-defined lower-level map for an outer learning problem, in the spirit of Antil, Di, and Khatri.

Third, shared-scene and orbital sensing. For a given trajectory, which scene patterns does the motion leave invisible? The null space of B sub SAR answers that question, and sparse orbital streams like the ISS data are a natural application.

Fourth, the edge implementation. The streaming form already updates each pixel in constant time. The next steps are updating only the pixels that receive events, and measuring time and power on a Raspberry Pi 5 and with sensors designed for the edge.

Each direction starts from something the dissertation already controls: an observation error, a solution map, a null space, or the structure of the matrix.

[If asked whether the learned-weight problem is well posed: Backup 67, page 97. If asked how the results extend to other regularizers: Backup 64, page 94, and Backup 27, page 57. If asked how the parameters are chosen now: Backup 66, page 96. If asked how sensor error reaches the estimate: Backup 36, page 66, and Backup 34, page 64.]

SLIDE 30

What the data constrain and what the models prove

[Read down each column: what the events constrain, then what the model proves.]

For temporal reconstruction, the events constrain brightness increments. For each fixed positive weight matrix, the regularized history is unique, with perturbation bounds, and the penalty selects the baseline.

[If asked about recomputed weights: Backup 65, page 95, gives a local Lipschitz bound beyond the dissertation. If asked how the theorems relate to the complete stereo estimator: Backup 30, page 60. If asked what is mathematically new: Backup 31, page 61, and Backup 64, page 94. If asked how close a stable estimate is to the truth: Backup 33, page 63. If asked what the events lose beyond the baseline: Backup 34, page 64. If asked whether the flow result is stable in the raw events: Backup 36, page 66. If asked whether a unique inner solve gives the right disparity: Backup 54, page 84.]

For flow, the events give directional constraints on the motion. Smoothness and integrated directional information together give a unique H one minimizer.

For stereo, the two cameras give paired event-feature profiles. At a fixed candidate disparity, with the weights frozen, the inner solve is unique. Choosing the disparity is a separate comparison across candidates.

The proof tools are classical. The contribution is the analysis of these event-derived models under explicit hypotheses.

[Point to the box.]

So each estimate is unique once the model supplies what the events leave free, under the hypotheses in the table: a unique reconstruction, a unique flow, and a unique stereo inner solve at each candidate disparity. Accuracy also depends on the representation.

Thank you. I'm happy to take your questions.

[Stop. Leave the closing slide visible. For questions: the discussion guide is Backup 01, page 31; flow detail starts at Backup 41, page 71; the stereo arc at Backup 47, page 77; references at page 110.]

BACKUP 01 / DECK 31

Discussion guide

[Navigation only. Choose the relevant technical question from this index; no spoken introduction is needed.]

BACKUP 02 / DECK 32

Must every temporal penalty entry be positive?

Likely question: Do you really need a positive penalty at every temporal sample?

[Point first to the kernel intersection, then to the connected-chain null space.]

Answer

No. The requirement is weaker: the observation operator and the penalty must not share a nonzero null vector. On the slide, that's the kernel intersection. Here A is the first-difference operator and Lambda is the penalty. The quadratic form of the reconstruction matrix is the squared norm of A times a vector, plus the squared norm of Lambda times that vector.

On a connected temporal chain, the null space of A is the constant profiles. So a single nonzero penalty entry is enough to remove the ambiguity. Fixing one reference value, or requiring zero mean, also picks out a unique answer. We use a fully positive diagonal in the theorem because it gives a simple quantitative lower bound along with uniqueness. A measured reference would go further: it would supply actual physical information about the brightness baseline.

If asked for more

Selecting the constant is different from regularizing the whole profile. A zero-mean constraint fixes only the unobserved offset. A magnitude penalty also damps the nonconstant modes and changes how they respond to noise. With a single anchoring entry, the matrix is still positive definite, but the lower bound based on the smallest diagonal entry has to be replaced by a bound for that anchored matrix.

BACKUP 03 / DECK 33

The event measure has a jump primitive

Likely question: What is the precise continuous-time interpretation of accumulating events?

[Indicate the right-closed interval convention, then the two primitives.]

Answer

At one pixel, we write the events as a signed measure: a point mass at each timestamp, weighted by its polarity. Its cumulative primitive is a step function. Start from a chosen initial value, and add the signed mass of the events up to the current time. Counting the right endpoint of each interval makes the primitive right-continuous, and its increments then match the event measure exactly.

With finitely many events, this primitive has bounded variation. Smoothing the measure with a standard kernel gives a regular temporal driver, and integrating that driver gives a smooth approximation of the staircase. As the kernel width shrinks, these smooth primitives converge in L one, for finitely many interior events. That describes the cumulative-event surrogate. The regularized reconstruction then makes its own, separate choice about fitting those increments and controlling the size of the profile.

If asked for more

A nonzero jump has a singular distributional derivative, so the unsmoothed primitive naturally lives in bounded variation, outside H one. At any positive kernel width, the kernels here give an L two driver and an H one primitive. On the discrete side, the first-difference matrix computes increments; dividing by the time step would instead approximate a derivative. Keeping that scaling and the interval endpoints consistent is essential when comparing the continuous and discrete descriptions. A joint limit in the grid and the regularization would need its own argument, beyond convergence of these primitives.

BACKUP 04 / DECK 34

Latent-signal gain and observation-noise gain are different

Likely question: What does regularization do to the signal and to measurement noise?

[Contrast the first and second table rows before discussing the variance row.]

Answer

They're two related but different transfer functions. On this slide, the penalty is a single positive scalar, lambda, and the observations are unweighted. Let g be an eigenvalue of A transpose A.

First, the signal. Take an underlying temporal profile, apply A to get ideal differences, and reconstruct. Each profile mode comes back multiplied by g over g plus lambda squared. The low modes are damped most, and the constant mode is removed.

Second, the noise. If instead we perturb the difference observations, the amplitude gain is the square root of g, over g plus lambda squared. For independent white noise with variance sigma squared, squaring that gain gives the reconstructed variance in each mode.

If asked for more

The singular value decomposition makes this explicit. A maps the right singular modes of the profile to the left singular modes of the observations, and the inverse maps back with the gain shown here. Treating g as continuous, the variance factor peaks at g equal to lambda squared, where it equals one over four lambda squared. On a finite chain the maximum can be smaller, because the eigenvalues are discrete. These exact formulas hold for the scalar, unweighted model; general diagonal or adaptive weights need the corresponding matrix analysis.

BACKUP 05 / DECK 35

Kernel performance depends on the scene and adaptive weights

Likely question: Why does changing the temporal kernel change both accuracy and conditioning?

[Point across the indoor and outdoor columns to show the change in ranking.]

Answer

Because the kernel changes the data, and the adaptive penalty is computed from the data. The first-difference operator stays the same. But the represented observations change with the kernel, so the penalty entries can change too. That changes the reconstruction matrix, and with it the condition number.

The table also shows that the kernels score close together. Indoors, the Gaussian gives a PSNR of 8.68 and the B-spline 8.66; outdoors, the rectangular kernel is highest at 7.31, and all four are within 0.2 dB. With the same sigmoid weights, their mean per-pixel condition numbers are 259 to 268, while the choice of weight rule moves it from about 2.4 to 265. So the kernel changes reconstruction quality a little, and the weight rule sets the conditioning. Conditioning measures how sensitive the linear solve is; accuracy also depends on the data, and on the bias that regularization introduces.

If asked for more

If A and Lambda were both held fixed, changing only the right-hand side would leave the condition number unchanged. The positive-definiteness theorem applies to every fixed positive penalty this produces. It guarantees the solve, while the table compares actual reconstructions. Those are scored against reference images, and the events still leave the absolute temporal offset to the regularization or the reference convention.

BACKUP 06 / DECK 36

What the reconstruction scores measure

Likely question: How is each reconstruction metric computed, and what do the scores establish?

Answer

Briefly, metric by metric.

PSNR starts from the mean squared pixel error against a reference image. It's ten times the base-ten log of the peak value squared over that error. **SSIM** compares local means, variances, and covariance, which captures luminance, contrast, and structure. Both are averaged over frames.

EPI correlates the gradient magnitude of the mean reconstruction with the normalized event-count map. **ERC** correlates the absolute change between consecutive reconstructed frames with the absolute bump-integrated event data.

TC takes the root-mean-square change between consecutive frames, divides it by the reconstruction's intensity range, and exponentiates the negative. **TG** averages the absolute frame-to-frame changes over pixels and frame pairs. **TV** averages the sum of absolute horizontal and vertical gradients over pixels and frames.

SC is the fraction of each frame's Fourier power inside a central low-frequency disk, averaged over frames. **CN** is the ratio of the largest singular value of each active pixel's matrix to its smallest, averaged over the active pixels.

These measure different things. Under these conventions, a constant sequence gets the best possible TC, one, and an ERC of zero. Smoothness scores reward a static output, so we read reference fidelity, event consistency, and conditioning together.

If asked for more

For ERC, both sequences are truncated to the same number of slices, flattened, and compared by Pearson correlation. EPI also uses Pearson correlation. Inputs with zero variance get zero correlation. In SC, an all-zero frame contributes zero power fraction, and the disk radius is one eighth of the smaller image dimension.

PSNR and SSIM depend on the normalization used for scoring, and CN describes the specified diagnostic matrix. The baseline choice and the regularization affect image agreement as well as smoothness.

BACKUP 07 / DECK 37

Can the reconstruction run in real time?

Likely question: Can the reconstruction run in real time, and on what hardware?

Answer

Yes, on an ordinary CPU. The table reports processing time divided by recording time, including binning, for the Chapter 2 estimator on one CPU node. For DAVIS data, one thread takes about four milliseconds per hundred-millisecond window, and the streaming solver returns a new frame every two milliseconds at 0.39 of real time with 1.4 megabytes of state. The 640 by 480 cup recording, at 2.3 million events per second, runs at real time on one thread for batch windows and at 1.3 for streaming; eight threads bring both to about a fifth of real time.

The code solves the Chapter 2 normal equations, with the final interval weight repeated at the terminal sample, and it agrees with the MATLAB reference to rounding error. It needs only NumPy and Numba, so the same code targets ARM boards such as a Raspberry Pi. Backup 78 covers memory and low-power choices.

If asked for more

The streaming recurrence uses the positive terminal diagonal, one plus the final interval weight squared. It matches the newest endpoint of the full mathematical batch solve on a growing prefix, with finalized interval observations. The batch code skips pixels whose net event sum is zero, and a nonzero history can have zero net sum, so compare streaming and batch on the same pixel set.

A moving window changes the left boundary and needs additional handling. The streaming path uses sigmoid weights and delta binning; symmetric kernels would require look-ahead to finalize bins. Quiet pixels with earlier activity still evolve under the recurrence. I also built a C++ and CUDA module in iniVation's DV software for live reconstruction using a general sparse solver.

[For operation counts, memory, and deployment tradeoffs: Backup 78, page 108.]

BACKUP 08 / DECK 38

The weak equation supplies the natural boundary condition

Likely question: Which boundary condition does the flow model use?

[Move from the variational equation to the interior equation and then the weak boundary trace.]

Answer

The natural one: homogeneous Neumann. We minimize over all of H one, so the boundary values are free. Taking the first variation gives the weak equation shown here, for every H one test field. B is the data coefficient matrix, b is the data forcing, and alpha sets the spatial smoothing.

The argument has two steps. First, test fields supported inside the domain give the differential equation in the distributional sense. Under the coefficient hypotheses, B times the solution and b are both in L two, so the solution's distributional Laplacian is in L two as well. That's enough regularity to define a weak normal trace of the gradient. Second, going back to general test fields shows that this normal derivative is zero on the boundary, in that weak trace sense. That's the right boundary statement on the Lipschitz domains the theorem uses.

If asked for more

The last step is the generalized Green's identity. Its boundary pairing must vanish against the trace of every H one test field. The trace map reaches the whole boundary trace space, so the normal trace is zero. For a fixed anisotropic diffusion tensor, the same argument applies to the flux: the normal component of D times the gradient is zero. That's a conormal boundary condition. The weak formulation uses the bounded tensor directly, so D never needs a classical derivative.

BACKUP 09 / DECK 39

Proof: control the mean and the fluctuation separately

Likely question: How exactly does the proof control the mean?

[Point to each line as its role is explained.]

Answer

Through m, the smallest eigenvalue of the integrated data matrix B. Write the velocity as its spatial mean plus a zero-mean fluctuation, w. The gradient of w is the same as the gradient of the full field, so Poincaré's inequality bounds the L two norm of w by the gradient norm. Smoothness controls the fluctuation.

Smoothness charges nothing for a uniform pan, so the data have to fix the mean. Applying the integrated quadratic form to the mean gives a lower bound of m times the mean's squared size. To connect this to the energy of the full field, write the mean as v minus w. Apply the square root of B and use the squared triangle inequality. That bounds the mean term by twice the data energy of v, plus twice a B-weighted fluctuation term. The upper bound on B and Poincaré's inequality control that last term, which gives the displayed inequality.

Now the energy controls the mean, the fluctuation, and the gradient, so it bounds the full H one norm from below. The bilinear form is bounded and coercive, so Lax–Milgram gives a unique weak solution. The weak equation is the first-order condition of this strictly convex energy, so its solution is the unique minimizer.

If asked for more

An admissible constant is c zero equals the smaller of two numbers: m over twice the domain's area, and alpha squared over one plus C P squared plus twice the area times beta B times C P squared over m. It comes from writing the squared H one norm as the area times the mean's squared size, plus the squared L two norm of the fluctuation, plus the squared gradient norm. For fixed alpha and beta B, this bound decreases linearly as m decreases.

BACKUP 10 / DECK 40

Stability follows by subtracting weak equations

Likely question: How do you get the stability bound?

[Point to the two terms in the bound.]

Answer

By subtracting the two weak equations and reusing one coercivity constant. Take two sets of coefficients with common bounds on the data matrix B and the forcing b, and a common positive lower bound m on integrated identifiability. Those shared bounds give one coercivity constant, c zero, that works for both problems.

Subtract the two weak equations, and test with the difference of the solutions, z. The left side controls c zero times the squared H one norm of z. The right side has two parts. One comes from the change in b. The other comes from the change in B, acting on the second solution. Cauchy–Schwarz, together with the bound R b over c zero on that solution, gives the final estimate.

So small changes in the coefficients produce controlled changes in the flow, as long as identifiability stays uniformly positive. The coefficients change because of timestamp jitter, binning choices, and noise events. The constants grow as m shrinks; the stripe scene on slide 26 comes close to that case.

If asked whether a stable estimate is close to the true motion

The theorem is about the minimizer of the chosen energy. How close that minimizer is to the true motion depends on two things: how accurately the represented coefficients describe the motion, and how much the H one penalty smooths across motion boundaries. The known-motion experiments measure that distance directly. In the stripe interior, the estimate is unique but mostly horizontal, and the normal-flow reference there is 2.12 pixels from the true diagonal motion.

If asked for more

Linking the bound to raw sensor errors needs an additional model of the representation error. If a density-dependent diffusion tensor is recomputed, the regularizer changes too, and its L infinity perturbation must enter the bound as well.

BACKUP 11 / DECK 41

Discrete flow: connectivity and sampled directions matter

Likely question: How is the flow computed, and when does the iteration converge?

[Point to the local two-by-two matrix, then to the two terms on the right.]

Answer

By simultaneous block-Jacobi sweeps on a pixel graph. They converge to the unique discrete minimizer from any starting field when the weighted sum of the sampled data blocks is positive definite.

Each node stores two velocity components. Its data matrix B describes the measured directions, and b carries the corresponding observations. A positive node factor, q, weights that local contribution. At node i, the matrix combines the local data with alpha squared times the weighted degree: the total weight linking the node to its neighbors. Solving this two-by-two system blends the pixel's own edge measurement with its neighbors' previous velocities.

Every node uses the same old field. Once all the new velocities are computed, we replace the field and repeat. Each sweep carries information from image edges one more pixel into textureless regions.

The theorem assumes the graph is connected, its edge weights are symmetric and nonnegative, its degrees and node factors are positive, and each data block is positive semidefinite.

If asked for more

The reason mirrors the continuous argument: the graph controls variation, and the sampled data control the remaining constant direction. The graph structure also gives the spectral condition this particular iteration needs; the next backup has that argument. The theorem applies when a representation's stencil and update schedule match these hypotheses.

If asked whether this is a finite-element scheme

For the voxel solver, exactly. Its three-by-three box mean is the bilinear Q1 stiffness stencil, which weights all eight neighbors equally, and evaluating the data blocks at pixels is nodal quadrature, a lumped mass matrix. Its reflected border padding reproduces the Q1 natural Neumann rows as well, so that scheme is Q1 finite elements with lumped mass, boundary included. The structure-tensor solver's four-neighbor stencil is the P1 stencil on a uniform grid of right triangles in the interior. Its replicated border halves the inward coupling at boundary pixels, which is the cell-centered finite-volume Neumann condition instead. The continuum coefficients in Appendix A.1.3, alpha squared h squared over three and over four, are these correspondences.

BACKUP 12 / DECK 42

Why block-Jacobi converges for this graph model

Likely question: Why does block-Jacobi converge for your discrete flow system?

[Explain the two signs in the energy identity before pointing to the spectral interval.]

Answer

The proof uses the graph structure of this particular system. The graph is connected, its edge weights are symmetric and nonnegative, and the sampled data blocks are positive semidefinite with a positive-definite sum. D holds each node's weighted degree and data block; R holds the couplings between neighbors. Positive alpha and positive degrees make D positive definite.

The key step is showing that both D minus R and D plus R are positive definite. Their quadratic forms are sums of nonnegative data terms plus squared differences, or squared sums, across edges. Connectedness narrows their possible zero modes to constant fields or alternating constant fields, and the data blocks rule those out. After scaling by D to the minus one-half, these two inequalities put every eigenvalue of the symmetric coupling matrix strictly between minus one and one. That's exactly what simultaneous Jacobi sweeps need in order to converge.

If asked for more

For D minus R, a zero contribution on every edge forces neighbors to agree, so on a connected graph the field is one constant vector. For D plus R, neighbors must be opposites. A bipartite graph allows an alternating copy of one vector; an odd cycle forces that vector to zero. Either way, the positive-definite sum of data blocks rules out any surviving nonzero vector. The Jacobi matrix is similar to the scaled symmetric matrix, so the two share a spectrum. This proof covers the simultaneous update schedule.

If asked how fast it converges, or why not CG or multigrid

Slowly on large textureless regions. Where the data blocks vanish, a sweep is Jacobi for the graph Laplacian; on a region N pixels wide its spectral radius is close to one minus pi squared over two N squared, so the number of sweeps grows like N squared. Jacobi was used because each sweep is an independent two-by-two solve at every pixel: matrix-free, fully parallel, and the classical Horn–Schunck iteration. The theorem makes the system symmetric positive definite, so conjugate gradients applies directly, with iterations growing like N rather than N squared; Chapter 3 also notes that Gauss–Seidel and SOR converge by Ostrowski–Reich. Multigrid is the standard fast solver for this class; Bruhn and colleagues use it for real-time variational flow. The time-surface code sweeps in place, Gauss–Seidel style, while the theorem analyzes simultaneous sweeps.

BACKUP 13 / DECK 43

Anisotropy needs a uniform ellipticity floor

Likely question: How can directional smoothing preserve the well-posedness theorem?

[Use the two eigenvalues to distinguish tangent smoothing from normal smoothing.]

Answer

The diffusion tensor needs a uniform positive lower bound on its eigenvalues. We use the normalized Nagel–Enkelmann form, built from the spatial gradient of event density. That gradient points across a density edge, and its perpendicular points along the edge. The tensor has eigenvalue one along the edge and a smaller positive eigenvalue across it.

So it penalizes flow changes across a density edge less, while still smoothing fully along it. The positive parameter gamma provides the floor. When the density gradient is bounded, the formula gives one positive lower bound, theta, over the whole domain. The coercivity proof then goes through with alpha squared times theta in place of alpha squared. So the directional model keeps the same existence and uniqueness argument under the integrated identifiability condition, with constants that depend explicitly on that floor.

If asked for more

A pointwise positive eigenvalue could still approach zero somewhere in the domain, which is why the proof needs a uniform floor. If the tensor is recomputed when the data change, the stability estimate must also account for the change in D, measured in the essential-supremum norm. The tensor follows event-density edges; treating those as motion boundaries is a modeling choice. The theorem guarantees solvability for the specified tensor.

BACKUP 14 / DECK 44

A variable diffusion tensor must stay inside the divergence

Likely question: Can the anisotropic operator be implemented by weighting the second derivatives?

[Point to the coefficient-derivative terms, then to the affine example.]

Answer

Only when the extra coefficient terms vanish, for example when the tensor is constant. For a spatially varying tensor, the divergence form contains both second derivatives of the field and first derivatives of the tensor coefficients; the product rule gives all the terms on this slide. Keeping only the tensor-weighted Hessian drops the coefficient terms.

The example makes this concrete. Take event density x squared over two, gamma equal to one, and the test field u equal to x. The tensor is diagonal, with first entry one over one plus x squared. The field has zero second derivatives, yet its flux changes with x, because that tensor entry changes. Differentiating the flux gives minus two x over the square of one plus x squared. A discretization has to capture that changing flux to approximate the stated model.

If asked for more

The weak formulation is useful when D is only bounded and measurable. It integrates the test-field gradient against D times the solution gradient, so coefficient derivatives never appear. A flux-based discretization can approximate this divergence structure directly.

BACKUP 15 / DECK 45

The regression radius formula is a heuristic

Likely question: Is the regression-radius formula an optimal choice of neighborhood?

[Identify the increasing bias term and decreasing variance term before discussing the radius.]

Answer

Only in a leading-order sense. It optimizes the leading approximation to the exact error bound for the actual lattice disk of sample points. The assumptions are a complete disk on a square lattice, a radius large compared with the lattice spacing, and a time surface that's smooth throughout the disk. L sub H measures how fast the Hessian can change, sigma is the timestamp-noise standard deviation, and h is the lattice spacing.

The leading squared-bias term grows like the radius to the fourth, and the variance term shrinks like its reciprocal. Balancing them gives the radius shown. Noisier data favor more averaging; faster-changing curvature favors a smaller neighborhood. We use that scaling as a principled guide. For an actual finite neighborhood, the exact lattice sums give the appropriate bound. And the whole fitting disk has to stay inside the region where the smoothness assumptions hold.

If asked for more

Minimizing one constant times r to the fourth plus another constant times r to the minus fourth makes r to the eighth equal to the ratio of the second constant to the first. Substituting the coefficients gives the factor shown. The asymptotic remainder can have either sign, so the leading expression approximates the finite-design bound from either side. And for an affine time surface the curvature bias is zero, so more samples only reduce variance, and this model has no finite balancing radius.

BACKUP 16 / DECK 46

Uniform and OLS weights agree through quadratics and differ on cubics

Likely question: Why must the temporal slope and spatial projection use compatible weights?

[Point to the common temporal weight and then the cubic residual.]

Answer

Because they enter the same transport equation, and that equation stays exact only if both use the same averaging. The ordinary least-squares temporal slope can be written as a weighted average of the time derivative. Its weight is the parabola shown here: zero at the endpoints and largest in the middle of the interval.

If we integrate the transport equation against that weight, the spatial term must use the same weighted projection. Pairing the OLS slope with a uniform temporal projection instead leaves the residual shown. For the transported cubic, that residual is one tenth, even though the underlying field satisfies transport exactly. So there are two separate questions: whether the coefficient fields give a well-posed problem, and whether they represent the physical transport relation exactly. The flow theorem answers the first, for any coefficients that meet its hypotheses.

If asked for more

Integrating by parts in the OLS formula produces the parabolic weight. Uniform averaging and that weight agree on the zeroth and first temporal moments, so low-degree tests reveal little. For transported polynomials up to degree two, the mismatch is zero, which makes the cubic the natural explicit example. If the resulting coefficient error is controlled in the norms of the stability theorem, and identifiability stays positive, that theorem bounds its effect on the flow.

BACKUP 17 / DECK 47

Mean-zero conditioning removes the unobserved mode

Likely question: Can a mean-zero formulation avoid poor conditioning as lambda becomes small?

[Point to the excluded k equals zero mode and the new smallest eigenvalue.]

Answer

Yes, in the scalar, unweighted model. Removing the constant temporal modes gives a condition-number bound that stays finite as lambda goes to zero. The first-difference operator sends constants to zero, so the right-hand side has zero mean in each view. The scalar system preserves the mean-zero subspace, so for positive lambda the solution already lies in it.

On that subspace, the smallest temporal eigenvalue is g one instead of zero. The sum and difference blocks have eigenvalues g k plus lambda squared, and g k plus lambda squared plus two gamma, with the constant mode left out. Their ratio gives the exact restricted condition number, and its denominator stays positive even at lambda equal to zero. So a mean-zero parameterization helps the algebraic solve, provided we state the convention for the unobserved brightness baseline.

If asked for more

The bound is uniform in lambda for a fixed temporal grid and fixed coupling. It still depends on the grid: g one behaves like pi squared over n squared for the unscaled chain, so the lambda-uniform bound grows quadratically with the number of samples. At lambda equal to zero the full-space matrix still has a constant null mode, so to use the restricted guarantee, a numerical implementation has to actually enforce the mean-zero restriction. General diagonal penalties or unequal weights need their own matrix bounds.

BACKUP 18 / DECK 48

The selection theorem has an exact ellipsoid interpretation

Likely question: Can you characterize exactly which competing stereo profiles beat the ideal match?

[Explain inside, boundary, and outside using the completed-square expression.]

Answer

Yes. For the candidate-local unweighted model, the competitors that beat the ideal match fill an ellipsoid in observation space. We hold the positive regularizer and the coupling fixed across candidates, and assume the ideal candidate's right profile equals the left observation. S is the monocular cost matrix, T is the coupling-discrepancy matrix, and P is their sum.

Completing the square writes the cost gap as the squared P-distance from the candidate profile to a center c, minus rho squared. P defines the metric, so the surface of equal cost is an ellipsoid, and the ideal match lies on its boundary. A competitor inside the ellipsoid costs less than the ideal match; one outside costs more. That's an exact geometric picture of the attenuation effect. It also gives a simple sufficient rule: any mismatch larger than two rho in the P-norm loses to the ideal match.

If asked for more

Let the mismatch be the left observation minus the competing right observation. The cost gap is its squared P-norm minus a linear term involving S. Cauchy–Schwarz in the P inner product bounds that linear term by two rho times the mismatch norm, so the gap is positive whenever the mismatch norm exceeds two rho. The exact ellipsoid test is sharper, because it uses direction as well as distance. Both statements concern the fixed, unweighted candidate cost before spatial aggregation.

BACKUP 19 / DECK 49

An observation perturbation bound can protect a disparity winner

Likely question: How much observation noise can the stereo decision tolerate?

[Indicate that the maximum includes the winner, then the half-margin condition.]

Answer

With a positive winning margin, we can put a number on it, for a fixed candidate list and fixed model parameters. We hold the left observation, A, Lambda, and gamma fixed, and perturb each right candidate profile. The cost is quadratic in that right observation: its gradient is two P times the displacement from c, and its Hessian is two P.

The operator norm of P is at most one half. Expanding the quadratic, each cost changes by at most the candidate's distance from c times the size of the perturbation, plus one half the perturbation size squared. Eta is the largest such bound over all candidates, including the original winner. If eta is less than half the original winning margin, every competitor stays more expensive. That protects the decision even though the formerly ideal right profile may be perturbed too.

If asked for more

The factor of two in the margin condition covers two changes at once: the winner's cost can rise by eta while a competitor's falls by eta. So the perturbed gap is at least the original margin minus two eta. For this observation bound, the quadratic expansion is exact. If the preprocessing, the left data, or adaptive parameters also change, their contributions must go into the score-perturbation estimate. The same margin argument then applies to whatever uniform score bound results.

BACKUP 20 / DECK 50

How the reciprocal IRLS contraction bound is obtained

Likely question: Where does the sufficient IRLS contraction constant come from?

[Follow the difference identity through the operator, residual, and weight estimates.]

Answer

We compare two frozen-weight solves generated by two input states. B time is the block-diagonal temporal difference operator for the two views. Subtracting the normal equations expresses the difference of the solutions as an inverse-system operator applied to a weight difference times a residual.

Three estimates control that expression. First, the positive regularizer bounds the inverse operator in terms of ell, the square of the smallest penalty entry. Second, comparing the frozen optimum with zero bounds its weighted residual by the weighted data norm. Third, reciprocal weights change in a controlled way: the change in an absolute value is at most the change in its argument, and every weight is at most one over epsilon. Combining the three gives q on the slide. When q is below one, the frozen-solve map is a contraction, and its iterates converge geometrically to a unique fixed point.

If asked for more

The weight estimate pairs with the weighted residual, and that's what produces the epsilon to the minus three-halves. The change in predicted temporal differences is at most the norm of A times the change in the state, which supplies the remaining factor. This is a sufficient global bound for the unclipped reciprocal map. It certifies contraction whenever that constant is below one.

BACKUP 21 / DECK 51

The frozen-weight fixed point and the likelihood stationary point differ

Likely question: Does convergence of the reweighted iteration imply a maximum-likelihood estimate?

[At the fixed point, show that the residual term vanishes while the log-variance derivative remains.]

Answer

No. When the variance depends on the unknown state, a frozen-weight fixed point and a likelihood optimum solve different derivative equations. A scalar example shows this directly. Take a positive observation y, with variance equal to the absolute prediction plus a positive epsilon. With the variance frozen, every least-squares step is minimized at the prediction y, so the iteration reaches that fixed point immediately.

The full Gaussian negative log-likelihood is different: the variance moves with the prediction, which adds a log-variance term. At the prediction y the residual is zero, but the derivative of the log-variance term is positive: one over two times the quantity y plus epsilon. So lowering the prediction slightly lowers the negative log-likelihood: its minimizer lies just below the fixed point.

If asked for more

This scalar example deliberately has no regularizer, to keep the mechanism visible. It isolates the weighting mechanism from the positive-penalty temporal theorem. The dissertation also gives a two-sample temporal example with positive regularization, where a fixed point differs from a stationary point of the final-state-weighted score, even before the Gaussian log term is added. The empirical estimator is evaluated as the specified surrogate procedure, with its own objective and stopping rules.

BACKUP 22 / DECK 52

What the implemented IRLS procedure returns

Likely question: What does one candidate-level IRLS call actually return?

[Walk through the five steps, emphasizing the different solve and scoring weights.]

Answer

It returns the last accepted latent iterate and its computed score. The procedure starts from the exact unweighted solve. At each iteration it forms predicted-rate weights, clips them for the linear solve, and freezes them. That system is positive definite. The algorithm solves it and takes a damped step toward the frozen minimizer.

For scoring, it recomputes the weights at the proposed state with the unclipped rule, then applies the acceptance and stopping tests. The score uses the weighted residual and the regularization terms, without a Gaussian log-variance term. So clipping, damping, and stopping are all part of the estimator being evaluated. After these candidate-level steps, masking, averaging, margin normalization, and semi-global matching adjust the candidate comparisons. The existence and contraction results describe the specified reweighting map; the experiments assess the complete implemented procedure.

If asked for more

A stopping rule can return an accepted iterate before it reaches a fixed point. Later, finite penalty values used for guidance or rejection go into averaging, so they act as soft preferences. The final winner is the exact minimizer of its computed score list, and path aggregation supplies the spatial context for that list.

BACKUP 23 / DECK 53

Depth evaluation distinguishes returned coverage and scored coverage

Likely question: Does WLS achieve its better scores by returning estimates at many more pixels?

[Contrast the first two numeric columns before discussing the scored-point totals.]

Answer

No. Both methods return estimates at about half the eligible pixels; the difference is in how many of those estimates fall inside the reporting band. The table keeps returned coverage separate from the population used to score accuracy. First, the ground truth must be finite and inside the band. Returned coverage counts finite estimates at those locations. Scoring also requires the estimate itself to fall between 0.5 and 8 meters.

The control returns finite estimates at 50.17 percent of eligible locations, and WLS at 49.48 percent. The in-band share of those estimates is where they differ: 51.54 percent for the control, and 96.40 percent for WLS. That gives 1.43 million scored points for the control, and 2.66 million for WLS. So WLS has a much larger scored population with comparable returned coverage. These are complete configurations, so the comparison includes their preprocessing, weighting, and final masking.

If asked for more

A finite depth outside the band is excluded by the evaluation convention, though it may still be physically possible. That's why returned coverage, in-band retention, and accuracy answer different questions. The benchmark's raw outputs include interpolation around the finest-level score minimum, and leave out the separate posterior refinement step. Reading the errors alongside these counts keeps the interpretation precise. A common-population comparison would score both methods on the same eligible pixels.

BACKUP 24 / DECK 54

Exact zeros alone produce large pooled kurtosis

Likely question: Does the large pooled residual kurtosis justify a heavy-tailed noise model?

[Point to the single active-probability factor in both moments and its effect in their ratio.]

Answer

On its own, it's a reason to look more closely at the residuals. This simple mixture shows why. Suppose a fraction of pixels have exactly zero residual, and the rest are zero-mean Gaussian with variance sigma squared. The Gaussian part has ordinary Gaussian tails. But pooling it with the point mass at zero gives an excess kurtosis of three over the active fraction, minus three.

So the more zeros, the larger the pooled kurtosis. Event-free pixels can have exactly zero fitted transport residuals, so sparsity alone can create the effect. Regularized fitting, unequal variances, and temporal dependence also shape the residual distribution. So I'd read the pooled diagnostic alongside the event-active residuals and how their variance scales. That separates concentration at zero from the tails of the active errors, and gives a better basis for choosing the observation model.

If asked for more

The moment calculation is direct. Only the active Gaussian part contributes to the second and fourth moments, so each picks up one factor of the active probability. Dividing the fourth moment by the square of the second leaves the reciprocal of that probability. A useful follow-up would report the zero fraction, standardize the active residuals by their variance, and compare with synthetic residuals passed through the same fitting procedure. Robust penalties remain an option, but choosing one needs evidence beyond this pooled moment.

BACKUP 25 / DECK 55

Adaptive temporal weights are fixed in the theorem

Likely question: How do the adaptive temporal penalties relate to the fixed-weight theorem?

[Contrast the activity-decreasing Chapter 2 rules with the energy-increasing Chapter 4 rule.]

Answer

The theorem applies once the adaptive rule has produced a positive penalty matrix. In Chapter 2, the sigmoid, inverse-proportional, and exponential rules all lower the magnitude penalty as event activity grows. That allows more temporal variation where there's activity, while a positive floor keeps every weight above zero.

Chapter 4 uses a different empirical rule, based on energy before normalization. Its weight increases with that energy, and it also has a positive floor. So the two chapters make different design choices. For every fixed positive matrix either rule produces, the quadratic solve is unique. But when the data change and the rule recomputes Lambda, both the data term and the penalty change. The sensitivity of the full adaptive reconstruction has to include both contributions, beyond the fixed-penalty data bound.

If asked for more

Where the adaptive rule is differentiable, differentiating the normal equations gives two forcing terms: A transpose times the change in the observations, and minus the change in Lambda squared applied to the current solution. Multiplying by the inverse reconstruction matrix turns those into the first-order change in the solution. Absolute values and clipping introduce kinks, but every rule is still Lipschitz, so the local bound on page 95 needs no derivatives. The positive-floor theorem is still useful, because it controls every individual solve the rule produces.

BACKUP 26 / DECK 56

A real MVSEC flow example is qualitative

Likely question: What does the real-data flow comparison establish?

[Identify the reference and ground truth first; compare methods only through visible features.]

Answer

It's a qualitative comparison on one frame: MVSEC indoor flying 2, frame 0340. The reference image shows the scene, and the ground-truth flow is the visual target. The other panels show the event-accumulation baseline and the time-surface, voxel, and structure-tensor voxel estimates.

Those last three are the representations covered by the common quadratic analysis. They build different coefficient fields from the same kind of event input, so the panels show how those choices change the flow pattern in a real scene. The color legend is the one from slide 8: hue is direction and brightness is size, with each panel scaled to its own ninety-fifth-percentile magnitude, so compare directions across panels and sizes only within a panel. The accumulation baseline is a separate comparison method. Chapter 3's endpoint errors come from the known-motion scenes, so here I describe what's visible in this frame.

If asked for more

Average endpoint error would measure the Euclidean distance between estimated and true velocity at a specified set of valid locations, then aggregate over a stated population. The theorem answers a different question: for coefficient fields meeting its assumptions, the regularized estimate exists, is unique, and depends continuously on the coefficients. Accuracy against true motion comes from evaluation, reported with its own valid set and aggregation rule.

The ground-truth flow has known limits too. It is generated from a Velodyne lidar with a 3D reconstruction, so it carries estimation and alignment error, and accumulating points over frames accumulates odometry drift that can make objects look inflated or shifted against the event data. The reference motion is also small: roughly 95 percent of it is under about three pixels, so it says little about fast motion.

BACKUP 27 / DECK 57

Quadratic H^1 regularization and TV answer different questions

Likely question: Why choose a quadratic H-one flow model rather than total variation?

[Point to the quadratic gradient term, then state its smoothing consequence before discussing TV.]

Answer

The quadratic model gives one explicit well-posedness analysis that covers all the event-derived representations. Its gradient penalty defines a bilinear form, and the integrated directional-information condition controls the constant velocity component. Together with the gradient term and Poincaré's inequality, that gives H one coercivity. Lax–Milgram then gives a unique solution, and subtracting weak equations gives the coefficient-stability estimates.

The tradeoff is that quadratic smoothing spreads out sharp motion boundaries. Total variation is attractive when preserving jumps matters most, and its natural setting is the space of functions of bounded variation. It would need its own existence, uniqueness, and stability analysis. Within the quadratic framework, the normalized directional tensor is another option: it reduces smoothing across selected density edges, while keeping a positive ellipticity floor and the same coercivity proof.

If asked for more

The integrated full-rank condition is sufficient for this particular quadratic H one argument. Replacing the gradient penalty with total variation changes how the functional grows, and which compactness and coercivity arguments are available. A squared-residual fidelity can be studied with TV, but the theorem has to address that functional directly. For the signed-linear energy written here, simply substituting TV can even make the energy unbounded below. Stronger fidelity assumptions can give strict convexity and uniqueness in a TV model.

BACKUP 28 / DECK 58

Frozen weights match the value; an MM majorizer also matches the derivative

Likely question: Is the reciprocal-weight iteration a majorization–minimization algorithm?

[Point to the omitted denominator derivative and explain the derivative-contact requirement.]

Answer

Only when derivatives also match. For majorization–minimization, the frozen objective would have to lie above the state-dependent score and touch it at the current state. For differentiable functions, touching also means matching derivatives. Here the variance proxy depends on the predicted signal, so the derivative of the full score has an extra term from the changing denominator.

The frozen quadratic matches the score's value at the current state, but its derivative treats that denominator as constant. So the term shown remains in the full derivative, and it's generally nonzero. That's why I analyze the method through its frozen-solve map: I prove that a fixed point exists, and that the iteration converges under a sufficient contraction condition. Those are precise properties of the map. A monotone-descent claim for a state-dependent score would need a separate surrogate inequality, or an acceptance rule that actually enforces descent.

If asked for more

If a differentiable surrogate majorizes the score and touches it at an interior point, their difference has a local minimum of zero there. So the derivative of the difference vanishes, and the derivatives must match. The extra variance derivative vanishes in special cases, such as a zero residual. The implemented acceptance rule is a different mechanism and can allow small cost increases, so describe its behavior through its actual thresholds.

BACKUP 29 / DECK 59

A Gaussian model gives an evidence interpretation of attenuation

Likely question: Is there a probabilistic interpretation of the attenuation in the stereo cost?

[Connect the conditional mean c to the ellipsoid center, then discuss the evidence ratio.]

Answer

Yes, for the Gaussian model with fixed common parameters. Read the magnitude and coupling penalties as a Gaussian prior on the two latent profiles, and assume independent unit-variance Gaussian observation noise. After integrating out the profiles, the right observation given the left is Gaussian. Its mean is c, the same shifted center as in the cost ellipsoid, and its covariance is the inverse of two P.

So the model predicts an attenuated right observation. For a scalar regularizer, that attenuation is diagonal in the temporal observation modes. Comparing the coupled model's evidence with the independent model's leaves the discrepancy term plus a constant that is the same for every candidate. Under these Gaussian assumptions, that explains both the shifted prediction and a possible discrepancy-based comparison.

If asked for more

Integrating a quadratic Gaussian model produces determinant terms as well as the minimized quadratic cost. With fixed common parameters, the determinants are the same for every candidate, which is what allows this comparison. The prediction operator has eigenvalues between zero and one and is a contraction in the P metric; a Euclidean contraction would need extra structure. The discrepancy alone would favor an ideal equal-profile candidate over distinct competitors, when the coupling is positive and A has full row rank. That's an alternative score, separate from the evaluated stereo method.

BACKUP 30 / DECK 60

The implemented estimator has several distinct layers

Likely question: How do the individual theorems relate to the complete stereo estimator?

[Follow the diagram in order; attach each guarantee to its specific box.]

Answer

Each guarantee attaches to one layer of the estimator. The estimator first rectifies and motion-compensates the events, then bins and normalizes them. For each candidate disparity, it runs the frozen-weight solve with damping and computes a score at the returned iterate. Spatial masking, averaging, normalization, and semi-global matching then adjust the candidate costs.

The final step picks a disparity from that score list, interpolating around the finest-level minimum, and converts a positive valid disparity to depth. Semi-global matching encourages spatial coherence by penalizing disparity changes along several image paths. The inner uniqueness theorem covers the frozen quadratic solve. The selection and margin theorems cover candidate decisions under their hypotheses. The benchmark tests all the stages together, so its results describe the complete estimator and its scored population.

If asked for more

A few distinctions inside those layers. Solve weights are clipped; scoring weights are recomputed without clipping. Finite guidance penalties go into averaging, so the preference they create is soft. Picking the minimum of the final finite list is exact relative to that list; whether it's the physically correct match is a separate question. Likewise, path aggregation is its own specified spatial algorithm. The reported benchmark includes score-minimum interpolation and leaves out the separate hole-filling and refinement stage.

BACKUP 31 / DECK 61

What does the dissertation contribute mathematically?

Likely question: What is the mathematical contribution if the proof tools are classical?

[Use one specific consequence, such as attenuation or integrated directional information, after listing the contributions.]

Answer

The contribution is building event-based inverse models and analyzing them with precise assumptions and consequences. For temporal reconstruction, I identify the observation null space, set up a transparent regularized inverse, and derive quantitative guarantees for the fixed-weight solution. For flow, I put several event-derived coefficient constructions into one quadratic framework, and prove well-posedness and stability under explicit directional-information conditions.

For contrast maximization, I separate the coercivity that the spline regularizer supplies from the nonconvex focus term, and prove existence for the full finite-dimensional objective. For stereo, the block structure leads to a profiled-cost decomposition, an explicit attenuation mechanism, selection and robustness conditions, and a separate analysis of the adaptive weights. Classical tools make those arguments possible.

If asked for more

Stereo is a good example of why this matters. A unique latent reconstruction at each candidate tells us how the inner solve behaves. The cost decomposition then shows that the candidate comparison also carries monocular regularization bias. The attenuation example exposes that effect, and the separation and margin conditions are sufficient for picking and keeping the right candidate. Similarly, the common flow theorem makes explicit which directional information each event representation must supply, so that one global regularizer determines both velocity components.

BACKUP 32 / DECK 62

Literature anchors and where to find the claims

Likely question: Which prior work underlies these models and where are your detailed arguments?

[Use the relevant literature item for the question; refer to the dissertation chapter for proof or protocol details.]

Answer

Here are the main connections. For image reconstruction, Rebecq and colleagues learn an event-to-video map; Zhang, Yezzi, and Gallego pose a flow-based image inverse problem; and Antil and Sayre combine events with frames in a bilevel model. Gallego and colleagues' survey covers the sensor and the field. Horn and Schunck give the classical variational flow framework, where a data constraint is coupled to spatial smoothness. Benosman and colleagues develop the timestamp geometry behind event-based visual flow. Weickert and Schnörr establish well-posedness for classical quadratic flow. Gallego, Rebecq, and Scaramuzza introduce contrast maximization, which motivates the separate nonconvex formulation. And the MVSEC paper describes the multivehicle stereo event dataset used for evaluation.

The particular models, assumptions, and results are developed in the dissertation. Chapter 2 covers temporal reconstruction; Chapter 3, the event-derived flow models and the contrast-maximization analysis; Chapter 4, the stereo formulation and selection analysis. The appendix specifies the evaluation protocol. The full proofs and experimental definitions are in the chapters.

If asked for more

The key distinction is between a framework I build on and a result proved for the models in this dissertation. For example, Horn–Schunck supplies the data-plus-smoothness structure, while the event-derived coefficients and their integrated identifiability assumptions determine how the present theorem applies. The time-surface literature motivates the timestamp geometry, and the crossing assumptions pin down the exact constraint used here.

BACKUP 33 / DECK 63

Stable estimates can still have regularization bias

Likely question: If the reconstruction is unique and stable, how close is it to the true signal?

[Point to the two error terms in order: bias, then propagated observation noise.]

Answer

Stability says how much the estimate moves when the observations change. Accuracy also depends on the bias that regularization introduces. Here we assume a fixed temporal grid, a positive scalar regularization parameter, and observations equal to the exact temporal differences of a reference signal plus additive error.

Substituting that model into the reconstruction formula gives the error identity shown. The first term is regularization bias: the penalty shrinks parts of the reference signal. The second is the observation error passed through the reconstruction operator. Increasing regularization affects both, typically trading more shrinkage for better noise control. The constant component deserves special mention, because temporal differences remove it completely: the penalty selects that baseline, while the observations determine the changes.

If asked for more

For a mean-zero reference signal on a fixed grid, exact noiseless difference data recover the signal as the regularization parameter goes to zero, because every nonconstant mode's gain approaches one. With persistent observation noise, a refinement statement needs joint control of the noise level, the grid spacing, and the regularization. Any mismatch between physical events and the increment model adds another observation-error term.

BACKUP 34 / DECK 64

The physical event stream loses more than one baseline

Likely question: Does the constant nullspace describe all the information lost by an event camera?

[Point to the amplitude inequality, then compare the constant and oscillating profiles.]

Answer

No. The constant null space is exactly the ambiguity of the discrete first-difference model. The physical threshold sensor loses more, because it reports only threshold crossings. This slide gives an explicit example, with the same initial reference for both signals.

The first log intensity is constant. The second oscillates sinusoidally around that value, with amplitude strictly smaller than the contrast threshold. Its change from the reset reference stays inside the threshold the whole time, so both signals produce the same empty event stream. Exponentiating either one gives a positive physical intensity. So even time-varying signals can share an event stream. This matters for reading the reconstruction theorem: it proves uniqueness for the regularized increment surrogate. Connecting that surrogate to the physical signal also takes assumptions about the threshold measurement and about what the representation retains.

If asked for more

An empty stream makes the ambiguity easy to see. In a nonempty stream, a perturbation can shift event times by moving threshold crossings. A perturbation supported away from the crossings preserves the schedule, provided a positive threshold margin remains throughout its support. That has to be checked for the specific signal and event reference.

BACKUP 35 / DECK 65

Integrated directional information is necessary for uniqueness

Likely question: Why is positive integrated directional information necessary for unique optical flow?

[Use the displayed energy identity to distinguish a nonzero linear slope from a flat direction.]

Answer

Take the quadratic energy on all H one vector fields, with a positive gradient penalty and a bounded positive semidefinite data matrix field. Suppose the integrated matrix is singular. Then it has a nonzero null vector, c, and positivity forces the data matrix to annihilate c almost everywhere.

Now add any multiple of c to a velocity field. Both quadratic terms stay the same: the gradient is unchanged because c is constant, and the data quadratic has zero curvature in that direction. Only the linear forcing term can change. If its coefficient along c is nonzero, choosing the sign of the multiple sends the energy down without bound. If that coefficient is zero, the energy is constant along the whole family. Either way there's no unique minimizer. So for this unrestricted model, uniqueness requires the integrated matrix to be positive definite.

If asked for more

The key step uses positive semidefiniteness pointwise. With B the data matrix, the nonnegative function c transpose B c has zero integral, so it's zero almost everywhere, and the square-root factorization then gives B c equal to zero. In the flat case, whether a minimizer exists at all is a separate question. Boundary constraints or a magnitude penalty can control the constant direction, and they change the uniqueness criterion accordingly. On a disconnected domain, the gradient leaves one constant per component, so each component needs its own positive-definite integrated matrix; a sum over the whole domain can hide one component's ambiguity.

BACKUP 36 / DECK 66

Coefficient stability is one stage of event-to-flow stability

Likely question: How does your stability theorem translate into stability with respect to the raw events?

[Point first to the coefficient-to-flow arrow, then to the preprocessing arrow.]

Answer

The theorem controls the second arrow in this diagram: how the velocity changes when the fitted coefficient fields change. The first arrow covers the event representation, neighborhood selection, and any fitting used to build those coefficients. An event-to-flow theorem would connect the two arrows quantitatively.

That needs two things. First, a metric on event streams. For example, moving one event by a tiny amount of time changes its timestamp only slightly, but how far apart the two streams are depends heavily on the metric. Second, a bound on how that event perturbation changes the coefficient fields, in the norms the variational theorem uses. With a Lipschitz bound for the preprocessing and a shared positive coercivity bound, the two estimates compose. That's a concrete extension: prove representation stability in a suitable event metric, then carry it through the quadratic solve, which is already controlled.

If asked for more

Take one positive unit event and shift its time by any nonzero amount. The difference of the two point masses has total variation norm two, while their Wasserstein-one distance is just the size of the shift. Hard temporal bins can jump when an event crosses a bin boundary; smoothing with a fixed bandwidth supports a different continuity analysis. That's why the metric and the representation have to be specified before claiming stability for the whole chain from events to flow.

BACKUP 37 / DECK 67

Refinement requires a choice of continuum scaling

Likely question: How should the temporal regularization parameter change when you refine the time grid?

[Point to the factors one over h and h before identifying lambda at spacing h.]

Answer

That depends on which continuum energy we want to approximate. This slide uses a smooth illustrative model with uniform spacing h. The observations are increments, so each one is h times a sampled derivative-like value, g, and the difference residual also carries a factor of h.

The target energy is one half the integral of the squared derivative residual, plus beta-time squared over two times the integral of the squared signal. The first integral becomes the squared increment residual over two h. The second becomes beta-time squared times h over two, times the sum of squared samples. Multiplying the whole discrete objective by h leaves its minimizer unchanged and puts it in the Chapter 2 form. Matching coefficients then gives lambda at spacing h equal to beta-time times h, where beta-time is the fixed continuum penalty strength. So for this target energy and this observation convention, the penalty scales linearly with the grid spacing.

If asked for more

This calculation matches coefficients; a convergence proof would also have to control the quadrature, the approximation spaces, and the data errors. Endpoint quadrature may add endpoint weights. Physical event data also raise separate choices: the smoothing bandwidth, and how the noise changes under refinement. Those can lead to a different scaling. The principle is to specify the continuum functional and the units of the observations first, then derive the discrete penalty from them.

BACKUP 38 / DECK 68

Strong smoothness selects one constant flow

Likely question: What does the optical-flow estimator approach when spatial regularization becomes very strong?

[Point to the limiting constant and identify the integrated matrix as the quantity controlling the mean.]

Answer

It approaches one constant vector. With the domain and coefficients fixed under the unified theorem's assumptions, increasing alpha makes spatial variation more and more expensive, because alpha squared multiplies the gradient energy. The integrated data select the constant: it's the inverse of the integrated matrix B applied to the integral of the forcing b.

That formula has a direct meaning. The data term pools all the directional constraints across the domain, and the constant balances them in aggregate. Positive definiteness of the integrated matrix makes that balance unique. The convergence is in H one, so it controls both the velocity field and its gradient. Very strong regularization therefore gives a spatially uniform compromise set by the data, with local velocity differences progressively suppressed.

If asked for more

For the proof, subtract the limiting constant and test the weak equation with the difference. The remaining forcing has zero integral, so Poincaré's inequality bounds its pairing by the gradient norm, which gives a gradient bound of order alpha to the minus two. Testing with constant vectors gives an integrated data balance. Positive definiteness of the integrated matrix then controls the mean of the difference, which completes convergence in H one.

BACKUP 39 / DECK 69

Transporting a scalar differs from conserving a density

Likely question: Why use scalar transport rather than a continuity equation for the event representation?

[Point to the product-rule term containing the divergence of velocity.]

Answer

The two equations make different assumptions about the represented quantity. Scalar transport says a pattern value stays constant along each moving trajectory. A continuity equation treats the quantity as a conserved density, so its value also changes when the flow compresses or expands space.

Expanding the divergence in the continuity equation gives the scalar transport terms plus the represented value times the divergence of the velocity. That extra term is the compression effect. The two equations agree when it's zero, for example in divergence-free flow. Event representations come from threshold-triggered observations and preprocessing, so treating them as a conserved physical density needs an extra model. In the dissertation, scalar transport is the constraint being analyzed. A conservation formulation would possibly need a source term for the creation or loss of events.

If asked for more

A one-dimensional example separates the two. Let the velocity equal the position, and let the represented value be the constant one. Its time and space derivatives are zero, so scalar transport holds. But the divergence of value times velocity is one, so source-free conservation has a residual of one. A source term of one would balance it. This shows exactly where the compression term enters.

If asked whether this could measure fluid flow

Event cameras are already used for particle velocimetry, for example by Willert and Klinner. Seeded particles give the flow a texture that moves with it, close to the scalar-transport model; conservation fits when the transported quantity is a density. Seeding also supplies texture in every direction, so the integrated identifiability condition is easy to meet. A known reference flow would test how well the representation describes the motion, and the new imaging lab makes those controlled acquisitions possible.

BACKUP 40 / DECK 70

Correlated errors change the uncertainty; coercivity holds

Likely question: What changes if the timestamp errors are correlated?

[Point to the covariance matrix in the sandwich formula, then to the positive regularizer in the temporal matrix.]

Answer

Correlation changes the uncertainty, through the full error covariance matrix. For the centered symmetric regression design on this slide, the slope estimate is a linear map of the timestamps. Applying that map to the covariance gives the sandwich formula shown: inverse Q, then the design transpose, the covariance, the design, and inverse Q again. With independent errors of common variance, it reduces to the variance times inverse Q.

The reconstruction question is separate. If a positive definite covariance matrix Gamma is used in a weighted temporal fidelity term, its inverse supplies the precision weights. Adding the positive magnitude penalty keeps the temporal system matrix positive definite. So the regularized solve is still unique, with stability constants that reflect the chosen weights. Correlation affects the uncertainty and the weighting model; coercivity comes from the positive quadratic terms.

If asked for more

A common error mean cancels from the slope when the design is centered, even with correlated errors. Smoothing changes the covariance to the smoothing operator times the covariance times its transpose. That matrix can be singular, so an inverse-covariance formulation needs either a positive definite covariance or a treatment of its null space. A Gaussian likelihood reading needs a Gaussian assumption on top of that; the covariance identity itself only needs finite second moments.

BACKUP 41 / DECK 71

Optical flow: further analysis

[Navigation only. Silent navigation cue: select the regression or contrast-maximization slide matching the question.]

BACKUP 42 / DECK 72

Time-surface gradient estimation: bias versus variance

Likely question: What determines the accuracy of your time-surface gradient estimate?

[Point to Q for sample geometry, then to the two contributions in the mean squared error.]

Answer

Two things: the geometry of the samples, and the curvature of the time surface. The estimator fits a plane to timestamps at nearby spatial offsets, and Q summarizes the geometry of those offsets. If Q is positive definite, the samples span both spatial directions and the fitted slope is determined. With a centrally symmetric design and independent timestamp errors of common mean and variance, the slope covariance is the noise variance times inverse Q.

The bias comes from fitting a plane to a curved surface. Central symmetry cancels the quadratic Taylor term in the slope. A Lipschitz bound on the Hessian controls the remaining cubic error and gives the bias bound shown, which holds for the actual finite set of samples. The mean squared error is then the squared bias plus the trace of the covariance. A larger neighborhood can improve the geometry and reduce the variance, but it also takes in more curvature. That's the bias–variance tradeoff here.

If asked for more

The design matrix depends on the sample positions. A well-conditioned fit can estimate the time-surface gradient accurately even at a moving straight edge, and the tangential velocity is still free, because one reliable normal constraint fixes only one component of motion. So regression rank and velocity identifiability answer different questions. The uncertainty formula also assumes an explicit timestamp-error model; a model for event counts would need its own derivation.

BACKUP 43 / DECK 73

Contrast maximization finds the motion that sharpens the events

Likely question: What does contrast maximization model, and why is it worth pursuing?

[Describe the warp on the left, then point to the CMAX column of the table.]

Answer

An edge moving across the sensor produces events all along its path during the time window, so a plain accumulation of those events looks like a smeared trail. Contrast maximization slides each event back along a candidate flow to one reference time. When the candidate matches the true motion, the trail stacks into a sharp edge. Image variance measures that sharpness, so the method searches for the flow that makes the warped event image sharpest.

The method works directly on individual events and their own timestamps. The quadratic models first summarize events as a time surface or in temporal bins, and an edge that moves during a bin blurs that summary. On the known-motion scenes in Chapter Three, contrast maximization has the lowest endpoint error in all three: 0.49 pixels against 2.15 for the structure tensor on idealized stripes, 1.19 against 1.92 on simulated stripes, and 0.32 against 0.98 on the simulated checkerboard. A zero flow field scores 3.00. On the stripes it recovers the diagonal motion, because warping events over the whole window brings in evidence from the object's boundary.

If asked for more

These numbers come from one representative window, chosen by the rule that minimizes deviation from each method's median error, on simulated scenes with known motion. On the idealized stripes, contrast maximization also assigns flow to the background around the object. The dissertation's endpoint errors come from these known-motion scenes. The guarantee for contrast maximization is that a global minimizer exists; the solver is a local quasi-Newton method. The contrast-maximization objective and existence backups give those details.

BACKUP 44 / DECK 74

How the known-motion comparison was evaluated

Likely question: How were the flow examples selected, and how fair is the comparison?

Answer

These are controlled tests of the geometry behind the models. A sixty-by-forty-pixel object translates diagonally by three pixels per evaluation window. The chapter uses idealized events generated directly, and events from the v2e simulator. Stripes emphasize the normal-motion constraint; the checkerboard supplies edge directions along both axes.

All the panels share one arrow scale and one selected window. Window twenty-two minimizes the summed deviation of each variant's endpoint error from that variant's median over windows one through thirty-one. AEE averages the Euclidean displacement error over the scored locations, in pixels per evaluation window. A zero estimate scores three pixels here.

The quadratic variants use alpha equal to twenty, without a pyramid; contrast maximization uses one hundred L-BFGS iterations. The implementations use different spatial stencils, so the same numerical alpha gives different effective smoothing. Their computational budgets also differ.

So the comparison illustrates directional information, and how these implementations behave at these settings. A dataset-level accuracy ranking needs its own evaluation and parameter-calibration protocol.

If asked for more

VOX uses an eight-neighbor interior stencil, and the structure-tensor implementation uses a conductance-weighted four-neighbor stencil. In the isotropic interior, their continuum smoothing coefficients are alpha squared times h squared over three, and over four, respectively. That describes the stencils only; the full models differ in other ways too.

The separate MVSEC contrast-maximization configuration caps iterations at ten; the controlled figures here use one hundred. The chapter calls the AEE evaluation mask M; the experiment's evaluator defines its exact population. The convergence theorem uses simultaneous block-Jacobi updates with fixed coefficients. The evaluated time-surface implementation uses in-place updates and event-dependent reference times; the convergence backup explains the schedule that's proved.

BACKUP 45 / DECK 75

Contrast maximization changes the optimization problem

Likely question: How does contrast maximization differ from your quadratic optical-flow model?

[Point to the warped locations, then identify the ridge lower bound in the regularizer.]

Answer

In contrast maximization, the event image itself changes as the motion estimate changes. Each event is warped to a common reference time, and well-aligned events produce a concentrated image. The objective rewards that concentration through image variance, and adds a quadratic regularizer on the motion coefficients.

The motion variable q is displacement over the normalized time window: physical velocity times the window duration. We represent that field with a tensor-product cubic B-spline on a uniform grid with eight-pixel knot spacing; the two flow components share the basis. The spline is continuous, so the field has a well-defined value at every event location, and the whole flow is a small coefficient vector rather than one unknown per pixel. Because the warp depends on the coefficients, the full objective is generally nonconvex. The regularizer does two separate jobs. Spatial diffusion controls variation across the field, and a positive coefficient ridge controls the overall size of the coefficients, including constant motion. So the existence argument uses finite-dimensional continuity and coercivity.

If asked for more

The basis is clamped at the image border with a Neumann boundary treatment, the natural condition for a rectangular frame. Concentration alone can also reward a wrong warp that collapses events together, which is why the regularization and the model restrictions matter. Continuous splines can be evaluated at points; a general two-dimensional H one field is an equivalence class, and needs more regularity before point values make sense.

If asked why analyze the quadratic family at all

Because the two answer different questions. The quadratic models are where I can prove what the events determine: a unique flow, stable in its coefficients, computed by an iteration that converges, under a condition you can check in the scene — edges in two orientations. That condition is what explains the stripe result. Contrast maximization is more accurate on these tests, but its objective is nonconvex: I can prove a global minimizer exists, and the solver is local. So one tells us when the data identify the motion; the other is a stronger estimator with weaker guarantees.

BACKUP 46 / DECK 76

A global minimizer exists, even though the solver is local

Likely question: What guarantees that contrast maximization has a global minimizer?

[Follow the lower bound from bounded focus gain to the positive quadratic ridge.]

Answer

Two facts: the focus term is bounded, and the ridge grows. There are finitely many events with fixed weights. Their total absolute weight bounds how much mass bilinear accumulation can put on the image. In particular, both the sum of absolute pixel values and the largest absolute pixel value are bounded by that total weight. Bounding the variance by the second moment gives the focus bound shown.

So the negative variance term is bounded below. Meanwhile, the positive coefficient ridge grows quadratically with the coefficient norm, so the full objective goes to infinity as the coefficients grow without bound. Bilinear accumulation is continuous in the warped locations, so the finite-dimensional objective is continuous too. Its relevant sublevel sets are compact, and the extreme value theorem gives a global minimizer. That proves existence for the model; the local optimizer used in the experiments returns whatever solution its numerical search reaches.

If asked for more

The continuity holds even when events cross pixel boundaries, because the bilinear tent weights join continuously. Differentiability is only piecewise; that matters for the numerical optimizer, while the existence proof needs only continuity. Coercivity relies on a strictly positive ridge for the fixed discretization. Under refinement, that control holds in function space: the ridge is a uniform L two penalty, which Backup 68 shows. The proof gives existence; global uniqueness, or global convergence of a local solver, would need additional structure.

If asked why the knots are uniform when events cluster at edges

Uniform knots keep the stiffness matrix and the ridge scaling simple, and the proposition assumes them. Events are already adaptive samples, so knots refined where events are dense, for example hierarchical B-splines, are a natural extension. The existence proof needs only a nonnegative partition of unity. For the uniform bound, weight each coefficient in the ridge by its basis integral, so it stays a lumped L two mass term.

BACKUP 47 / DECK 77

Stereo depth: full Chapter 4 arc

[Navigation only. Silent navigation cue: choose the matching model, selection, IRLS, or benchmark page.]

BACKUP 48 / DECK 78

Stereo depth in one slide: what Chapter 4 adds

Likely question: What does Chapter 4 prove beyond the inner solve?

[Move through the numbered results deliberately.]

Answer

Three things: what the candidate cost actually compares, when the correct candidate wins, and when the reweighting converges.

The model takes one temporal profile per view and adds a soft penalty on their difference. For fixed nonnegative observation weights, positive magnitude weights, and nonnegative coupling, the temporal coercivity proof carries over directly, so every frozen inner problem has a unique solution.

The correspondence analysis looks at the unweighted cost for a single candidate, before spatial aggregation. After minimizing over the profiles, that cost has a monocular term for each view plus a coupling discrepancy. The right-view term changes with the candidate. In the exact example, the perfect match costs one third, while an attenuated candidate costs nine fortieths: a right view that's a weaker copy of the left can beat the true match, so the cost favors quieter candidates.

A separation condition guarantees that this model picks the correct candidate. Once the winner has a positive margin, cost errors each smaller than half that margin, say from noise events or mismatched thresholds between the two sensors, leave the winner unchanged.

For the reciprocal reweighting map, a fixed point exists. Under a sufficient contraction condition, it's unique and the iteration converges.

If asked for more

On the MVSEC benchmark, WLS leads on indoor flying two and narrowly on the aggregate log RMSE, while scoring more pixels than the local adaptation of Piatkowska and colleagues' method. Each method is scored on its own valid pixels. The separation results concern unweighted candidate-local costs before aggregation.

BACKUP 49 / DECK 79

Stereo alignment after motion compensation

Likely question: Why should two pixels' observed event signals agree when the rig and scene are moving?

Answer

Because, for a proposed correspondence, both pixels watch the same scene point once we account for the motion. The comparison assumes synchronized streams, calibrated rectification, and a scene point visible in both cameras.

The preprocessing also compensates motion within each slice. It fits one global translation separately in each camera and warps the events to the end of the slice. Subtracting the two warped horizontal coordinates gives the equation shown: the original disparity picks up a correction equal to the time to the reference, times the difference of the two horizontal warp velocities.

When the warps agree, the comparison is preserved. A horizontal mismatch changes the disparity the matcher sees; a vertical mismatch moves the correspondence off its row. The reported slices are twelve and a half milliseconds wide, within a quarter-second evaluation window.

The inner positive-definite system gives a unique fit to whatever observations it receives. Whether that fit reflects the physical correspondence also depends on the timing, calibration, and motion approximation used to build the observations.

If asked for more

For simultaneous points that start on the same row, subtracting the warped vertical coordinates gives the time to the reference, times the difference of the vertical warp velocities. A global translation approximates each slice; depth-dependent parallax, changing disparity, and independently moving objects can introduce mismatch.

These translations come from Chapter 4's own preprocessing fit; Chapter 3's dense-flow estimator is a separate study. Soft coupling allows the profiles to disagree, but its penalty still favors agreement. Occlusion removes the visible match, which needs a rejection or visibility treatment beyond the unique inner fit.

BACKUP 50 / DECK 80

At each candidate, couple two temporal profiles

Likely question: What are the unknowns in the stereo reconstruction model?

[Point to the two data terms, two magnitude terms, and coupling term in that order.]

Answer

For a fixed left pixel and candidate disparity, the unknowns are two temporal feature profiles, one per view. The left observation is fixed, and the right observation is sampled at the candidate disparity. Each profile has its own weighted data-fidelity term, with the same temporal first-difference operator.

There are also two magnitude penalties and a soft coupling term. The magnitude penalties control the temporal baselines, and the coupling penalizes disagreement between the profiles, with gamma setting its strength. During a frozen solve, the weights and the positive diagonal regularizer are fixed, and gamma is nonnegative. The right profile is an auxiliary variable for this left pixel and candidate; each candidate gets its own right-profile solve. Spatial context comes in later, when the candidate costs are aggregated. Because preprocessing transforms the observations, we interpret these unknowns as event-feature profiles.

If asked for more

Setting both weight matrices to the identity gives the unweighted model used in the explicit selection analysis. With finite coupling, the profiles can differ, at a cost, so the data terms keep information from each view. Absolute calibrated log intensity would need a calibrated observation model linking the processed features to physical units; the candidate-local optimization and its positive-definiteness proof concern the stated feature model.

BACKUP 51 / DECK 81

Frozen weights give a unique, stable inner solve

Likely question: Why is the stereo solve unique when the weights are frozen?

[Point to the magnitude terms as the source of the uniform positive lower bound.]

Answer

Because with the weights frozen, every direction has positive curvature. For a fixed disparity, fixed positive weights, a positive diagonal regularizer, and nonnegative coupling, the objective is quadratic in the two latent profiles. Its Hessian is the block matrix shown. To show it's positive definite, evaluate its quadratic form on an arbitrary pair of perturbation profiles.

The result is a sum of two weighted difference energies, two magnitude-penalty energies, and the coupling energy of their difference. Every term is nonnegative. More than that, the magnitude terms alone are at least the smallest regularizer entry squared times the squared norm of the stacked perturbation. So every nonzero direction has strictly positive curvature. That gives exactly one global minimizer of the frozen quadratic, and an invertible linear system. Subtracting two such systems also gives stable dependence on the observations, as long as the matrix and weights stay fixed.

If asked for more

The inverse matrix has norm at most one over the smallest regularizer entry squared. To turn that into an observation-to-solution bound, also include the norm of the weighted observation map on the right-hand side. This theorem is about each inner solve. Candidate selection adds a discrete comparison, and changing the weights changes the quadratic itself; those stages have their own separation and fixed-point analyses.

BACKUP 52 / DECK 82

Sum and difference modes expose the coupling

Likely question: What does increasing the stereo coupling strength actually do?

[Point to K in the mean block and K plus twice gamma in the difference block.]

Answer

Coupling acts only on the disagreement between the views. When the two temporal blocks are equal, an orthogonal change of coordinates separates the common profile from the difference profile. Adding the normal equations cancels the coupling term; subtracting them adds two gamma to the temporal precision. So coupling suppresses left-right disagreement and leaves the equation for the mean unchanged.

The same decomposition gives an exact condition number in the unweighted case with a scalar magnitude penalty. The smallest full-space eigenvalue is lambda squared, from the shared constant temporal mode. The largest is the largest temporal difference eigenvalue plus lambda squared plus two gamma. Their ratio is the condition number shown. So at fixed lambda, increasing gamma raises the full-space condition number, even as it pulls the views into closer agreement. The mean profile still needs the magnitude penalty to control its baseline.

If asked for more

The orthogonal transform preserves Euclidean eigenvalues and condition numbers. The simple block diagonalization needs equal temporal blocks; unequal adaptive weights generally mix the mean and difference coordinates. A mean-zero restriction removes the shared constant mode for suitably structured perturbations, and can give a sharper bound. Full-space conditioning still matters, because numerical errors can enter in any direction, so it's useful to keep the two settings separate.

BACKUP 53 / DECK 83

Profile out the latent variables to see what disparity compares

Likely question: What quantity is actually compared when you choose a disparity?

[Point to the candidate-dependent right monocular term before the discrepancy term.]

Answer

In the unweighted model, we can solve out the two latent profiles exactly. The minimized cost splits into a left monocular cost, a right monocular cost, and a discrepancy term between the two observations. Here m is the minimum regularized temporal reconstruction cost for a single observation vector.

The left monocular term is the same for every candidate. The right monocular term changes with the candidate, and so does the discrepancy. The discrepancy is a nonnegative quadratic in the difference of the observations, passed through the transpose of the temporal operator. Its matrix is the inverse temporal precision minus the inverse precision with two gamma added. This exposes the selection mechanism: a candidate is rewarded both for matching the left observation and for having a low reconstruction cost of its own. Those two pulls can compete, which is why correspondence selection needs its own analysis.

If asked for more

The derivation completes the square in the stacked latent variables, then uses the sum-and-difference transform to separate the terms. The inverse difference equals two gamma times the product of the two positive inverse matrices, so it's positive semidefinite. An exact winner of these unaggregated profiled costs minimizes the corresponding joint objective over the candidates and the latent variables. Showing that it's the physical match takes assumptions that connect that objective to the true correspondence.

BACKUP 54 / DECK 84

An attenuated match can cost less than the perfect one

Likely question: Can the stereo objective favor a wrong correspondence even when the true profiles match exactly?

[Point to the exact-match point, the one-half point, and their displayed costs. Read the follow-up scalar factors slowly.]

Answer

Yes. This two-sample example shows exactly how. The temporal operator takes one difference, both magnitude penalties are the identity, and gamma equals one. The left observation is one, and the right candidate observation is a scalar, b. Solving the latent problem gives the quadratic cost in b shown here.

At the perfect match, b equals one and the cost is one third. At the attenuated candidate, b equals one half and the cost is nine fortieths, which is smaller. Over all b, the minimum is at two sevenths. Attenuation lowers the right monocular cost by more than it adds in discrepancy. Every inner solve here is still strictly convex with a unique solution. So the example isolates the selection problem: inner uniqueness gives a profile for each candidate, and whether the correct candidate wins depends on how their minimized costs compare.

If asked for more

For an observation scaled by b, the monocular part is one plus b squared, times the left monocular cost. The discrepancy part is one minus b, squared, times the discrepancy coefficient. The derivative at b equal to one is twice the positive monocular cost, so decreasing b slightly lowers the score. This concerns the unweighted local cost before aggregation; the evaluated weighted method has additional steps.

BACKUP 55 / DECK 85

Correct disparity needs a separation condition

Likely question: What assumptions ensure that the correct disparity wins?

[Subtract twice the left monocular cost and point to the nonnegative monocular-cost difference and the strictly positive discrepancy.]

Answer

A sufficient condition is that every competitor costs at least as much to reconstruct on its own as the left view does. The result concerns a finite candidate set and the unweighted local cost. The positive regularizer is shared by both views and all candidates, and the coupling strength is fixed and strictly positive. At the true candidate, the right observation equals the left observation exactly.

For every competitor, assume its right observation differs from the left, and its own monocular reconstruction cost is at least the left monocular cost. Subtracting the true candidate's cost leaves two parts. The monocular-cost difference is nonnegative by assumption. The discrepancy is strictly positive for a distinct observation, because the temporal difference matrix has full row rank and gamma is positive. Their sum is positive, so every competitor costs more, and the correct candidate wins uniquely. The condition works by ruling out the cost advantage that attenuation provides in the counterexample.

If asked for more

The condition is sufficient, and it can be stronger than necessary. A competitor with a smaller monocular cost can still lose if its discrepancy is large enough. The more general metric-separation criterion covers that case. Both versions concern ideal matched profiles and candidate-local, unweighted costs. Applying the conclusion after masking, adaptive weighting, or spatial aggregation requires checking that scoring procedure and its assumptions.

BACKUP 56 / DECK 86

A winning margin turns cost stability into selection stability

Likely question: How much can candidate costs change before the selected disparity changes?

[Point to the two eta contributions and the strict half-margin condition.]

Answer

By just under half the winning margin. Start with a fixed finite candidate set and a unique winner. Delta is the smallest gap between the winner and any competitor. If every candidate cost changes by at most eta, the worst case lowers a competitor by eta and raises the winner by eta, so the gap shrinks by at most two eta.

So whenever eta is strictly less than half of delta, every perturbed gap stays positive and the same candidate wins. That turns a bound on continuous cost perturbations into a statement about a discrete decision. It also shows why a small margin is fragile. If two right candidates have exactly the same temporal observations, and use the same local procedure and parameters, their local costs are equal. That tie can happen even with rich temporal activity; selection stability requires a positive gap between the candidate scores.

If asked for more

The cost bound must cover the original winner as well as every competitor. At exactly half the margin, the bound allows a tie, so the strict inequality matters. An observation-based estimate can supply eta when the score parameters are fixed and the observation perturbations are controlled. Spatial aggregation can separate candidates using neighboring evidence, so the identical-profile example applies to the local score.

BACKUP 57 / DECK 87

Depth sensitivity grows with distance

Likely question: What does a pixel of disparity error mean for depth accuracy?

Answer

It depends on distance: the same pixel matters much more far away. Depth is focal length times baseline divided by disparity. With fixed calibration, the exact change under a disparity perturbation is the difference formula on the slide. For a small relative disparity error, the absolute depth sensitivity is depth squared over focal length times baseline. So farther points turn the same disparity error into a larger depth error.

The examples hold calibration fixed and add one pixel of disparity. Going from four to five pixels underestimates depth by twenty percent. Going from twenty to twenty-one underestimates it by about four point seven six percent. These are illustrative calculations.

That's why we read depth error together with the depth range, the disparity precision, and the scored population. A stable winning label is one stage; interpolating it and converting it to depth each add their own sensitivity.

If asked for more

The approximation needs the disparity error to be small relative to the disparity; the exact identity holds for any positive original and perturbed disparities. If the calibration also changes, the first-order relative depth change is the relative focal-length change plus the relative baseline change, minus the relative disparity change.

On the benchmark, finite in-band estimates and ground truth define each method's scored population; the coverage slides explain that selection. This calculation explains how disparity errors propagate within those populations. Attributing a method's error tails would take a separate analysis of its returned disparities.

BACKUP 58 / DECK 88

Heteroscedastic weights: statistical motivation and scope

Likely question: Why do you use reciprocal activity weights in stereo?

[Point to the rate sum in the variance, then to the predicted-activity proxy in the weight.]

Answer

Because the noise model treats busy intervals as noisier. A useful starting point is a raw-count model where the positive and negative event counts are independent Poisson variables. Their difference has mean equal to the difference of the two rates, and variance equal to their sum. In a high-count approximation, inverse-variance weighting then motivates a weighted least-squares fidelity term.

The implemented rule uses reciprocal predicted activity plus a positive noise floor. It's a practical variance proxy: high predicted activity gets less weight, and the floor keeps the weight finite near zero. Total and net activity differ here: opposite polarity rates can cancel in the mean, while both still add to the variance. Also, the actual observations have been warped, downsampled, and normalized. So we treat each frozen weighted solve as a Gaussian surrogate for the processed features, with the reciprocal rule supported by the reported weighting comparison.

If asked for more

The absolute difference of the two polarity rates is at most their sum, with equality exactly when at least one rate is zero. So a variance based only on the net rate adds an approximation. In the iteration, the weights are evaluated at the previous profile and held fixed during the quadratic solve. That keeps the positive-definiteness argument for each solve, and makes the statistical approximation explicit.

BACKUP 59 / DECK 89

IRLS: fixed-point existence and conditional convergence

Likely question: Does the iteratively reweighted stereo solver converge?

[Point to the invariant-ball bound for existence, then separately to q less than one for convergence.]

Answer

Under an explicit condition, yes. The analysis gives two levels of guarantee for the unclipped reciprocal map, with the disparity, regularizer, and coupling fixed. First, a fixed point exists. Compare each frozen minimizer's objective with its value at zero. The positive magnitude penalty controls the solution's squared norm, and every reciprocal weight is at most one over the noise floor. Together, those put every update inside a sufficiently large closed ball.

The weight map is continuous, and the frozen system stays positive definite, so the update map is continuous. Brouwer's theorem then gives a fixed point. Second, the bound q shown is a sufficient contraction constant. If q is less than one, the fixed point is unique and the updates converge geometrically. That's the explicit parameter condition. All of this describes the analyzed update map; the implementation's stopping and acceptance rules decide when the numerical iteration ends.

If asked for more

With a fixed damping parameter greater than zero and at most one, the update is a convex combination of the old state and the undamped update. The fixed points stay the same, and the Lipschitz bound becomes one minus omega, plus omega times q. When q is below one, that's still a contraction. A fixed point means consistency with the frozen weights; reading it as a likelihood optimum needs a separate analysis of the objective.

BACKUP 60 / DECK 90

Stereo benchmark: read accuracy alongside scored coverage

Likely question: How competitive is the stereo method, and how should I interpret the coverage numbers?

[Identify the centimeter columns, the scaled log metric, and the scored-point column separately.]

Answer

It's competitive. Its mean errors are within six centimeters of the Piatkowska adaptation's on every sequence; it leads on Flying 2 and narrowly on the averaged log error; and it's scored on more than twice as many points. The table reports mean absolute depth error in centimeters on four MVSEC indoor sequences, plus an averaged log error and the total number of scored points. Soft-coupled weighted least squares has mean errors of 73.83, 58.35, 70.33, and 59.24 centimeters. Its displayed log RMSE is 44.54, on 2.66 million scored points.

The Piatkowska adaptation has the smaller mean error on the other three sequences. The other three prior methods, Xie, Kogler, and Rogister, trail weighted least squares on every sequence and on the log error. The population sizes matter: the adaptation is scored on 1.20 million points. For each method, a pixel is scored only when both its estimate and the ground truth fall inside the reporting band. So the table compares accuracy together with scored coverage. A direct accuracy comparison would score both methods on the same pixels.

If asked for more

The reporting band is 0.5 to 8 meters. Point counts are summed across sequences. The displayed log RMSE is one hundred times the base metric, averaged equally over the sequence-level values. These are complete implementations, including normalization, regularization, guidance, and activity masking, and the external methods are local adaptations.

The reference depth has known limits. MVSEC's ground truth comes from a Velodyne lidar combined with a 3D reconstruction, so it carries estimation and alignment error. Accumulating lidar points over many frames also accumulates odometry drift, which can make objects in the reference look inflated or shifted relative to the image and event data. Every method is scored against the same reference, so these errors limit how finely we can rank close results, such as differences of a few centimeters.

BACKUP 61 / DECK 91

A weighting comparison supports the selected rule

Likely question: What empirical evidence supports the reciprocal weighting rule?

[Point to the subtraction direction before reading the four positive differences.]

Answer

A controlled swap of the weighting rule, with everything else in the configuration held fixed. Each number is the rate-proportional method's reported mean depth error minus the reciprocal-weight method's. The differences are 4.44, 8.28, 2.73, and 5.17 centimeters on Flying 1 through Flying 4. All four are positive, so the reciprocal rule has the smaller reported mean error on every sequence.

That supports the selected rule for these configurations and their scored populations. Each method's evaluation population is defined separately, so the comparison combines its errors with the set of in-band points it returns. These sequences were also used to choose the rule. So I'd call this empirical support for the design choice; a common-pixel evaluation and additional sequences would test how far the advantage extends.

If asked for more

These are differences of reported mean errors. The empirical improvement motivates the implementation choice; a maximum-likelihood interpretation would also need a justified probabilistic model for the processed observations.

BACKUP 62 / DECK 92

Hard fusion changes both the constraint and the regularization

Likely question: Why does hard fusion perform differently from soft coupling?

[Compare the magnitude-penalty coefficients after dividing the hard-fusion equation by two.]

Answer

Hard fusion changes two things at once. It forces the two profiles to be equal and solves for one shared profile, and the implemented system also uses a different effective magnitude penalty. Dividing its normal equation by two shows half of lambda squared next to the temporal data matrix. By contrast, sending the soft model's coupling strength to infinity, with equal fixed weights and the same regularization, keeps the full lambda squared in the common-profile equation.

Averaged over the four sequences, the reported mean depth error is 102.10 centimeters for hard fusion and 65.44 for soft weighted least squares. That's the observed difference between the two evaluated configurations. But the comparison changes hard equality, effective regularization, weights, iteration budgets, and scored populations together. So it motivates controlled comparisons to find how much each choice contributes to the gap.

If asked for more

The soft model's mean equation already combines both views.

BACKUP 63 / DECK 93

Motion-induced events and the aperture problem

Likely question: Can you derive the aperture problem directly from the event-generating image model?

[Point to the normal-component equation, then to the arbitrary tangential coefficient in the boxed velocity family.]

Answer

Yes. Take a straight edge under steady illumination. Its log intensity is a one-dimensional profile evaluated at the normal position minus the normal speed times time. The spatial gradient points along the edge normal, and the time derivative is minus the normal speed times the profile's derivative.

Substituting those into brightness constancy gives the normal vector dotted with the velocity equal to the edge's normal speed, wherever the profile's derivative is nonzero. Every velocity with that normal component satisfies the same equation, whatever its tangential component. These velocities produce the same local log-intensity histories, and so the same ideal threshold events when their reset references agree. That's the aperture ambiguity in this model: the edge fixes its normal motion and leaves the tangential motion free. This assumes the edge's endpoints and any other distinguishing features lie outside the observation window.

If asked for more

If the normal speed is zero, the motion is purely tangential, and the pixel values stay constant in the ideal edge interior. With nonzero normal speed, the crossing-time surface has gradient equal to the normal divided by that speed, which recovers the time-surface transport constraint. Steady illumination is assumed to isolate motion. The sensor's threshold rule also responds to changes in illumination, so treating every event as motion needs additional scene assumptions.

BACKUP 64 / DECK 94

One complementarity lemma behind the uniqueness results

Likely question: The three chapters come from separate papers. Is there one mathematical mechanism behind their uniqueness results?

[Point to the three hypotheses of the lemma, then read the table one row at a time.]

Answer

Yes. Each problem has two quadratic terms. One controls every direction except a small, finite-dimensional set of modes, and the other has to see those modes. If both happen, the sum is coercive, and the coercivity constant falls linearly with how well the second term sees the leftover modes.

In the temporal problem the data see every temporal variation, but they miss the constant baseline; the penalty fixes it. One nonzero weight is enough. Flow reverses the roles. Smoothness controls every variation, but it charges nothing for a uniform pan, so the data have to see the pan; that's the integrated matrix condition. The discrete flow theorem is the same statement on a connected graph. In the stereo inner solve, the two data terms and the coupling leave only a common constant, and the penalty fixes it.

The condition is also necessary. If a leftover mode is invisible to the second term, the quadratic form vanishes there. Backup 02 and Backup 35 are exactly those two cases.

If asked for more

The proof is one line. Split u into its part in the leftover modes, n, and the rest, w. The square root of a is a seminorm, so a of n is at most twice a of u plus twice beta times the norm of w squared, and r controls the norm of w. Adding the two bounds gives the constant.

The constant explains the scene statements: for flow it falls as m shrinks, which is why the stripe scene is nearly degenerate. Two parts of the dissertation sit outside this lemma. Correspondence selection compares finitely many candidate costs, so it needs a separation margin. Contrast maximization is nonconvex, so compactness gives existence only. This lemma and Lax–Milgram apply to quadratic forms; total variation needs separate tools.

If asked how the result extends to another regularizer

The lemma is the recipe. First find the modes the new regularizer leaves free; then check that the data see them. A fractional seminorm on a bounded domain still leaves only the constants, so the same integrated condition fixes them, with a fractional Poincaré inequality and coercivity in H s. A nonquadratic penalty such as total variation needs its own existence argument, through compactness in BV.

BACKUP 65 / DECK 95

Adaptive weights keep the temporal estimate locally Lipschitz

Likely question: The temporal theorem fixes the weights, but you compute them from the data. Is the estimator you actually run stable?

[Point to the subtracted normal equations, then to the two terms of the bound.]

Answer

Yes, it's locally Lipschitz, by a short argument. Each Chapter 2 rule maps event activity to a weight between a positive floor and a finite ceiling, and each rule is Lipschitz in the activity. The absolute value, the minimum and the maximum are all Lipschitz, so their kinks need no differentiability.

Subtract the two normal equations. The difference in the solutions equals the fixed-weight term from the change in the data, plus the change in the squared weights acting on the second solution. Three standard bounds finish it. The solution operator has norm at most one over twice the floor. The inverse system matrix has norm at most one over the floor squared. Each rule sets the squared weight directly, so the squared weights change by at most L times the change in activity.

The second term grows with the size of the data, so the bound is Lipschitz on bounded sets of data. The table gives the floor, ceiling and Lipschitz constant for the sigmoid, inverse-proportional and exponential rules.

If asked for more

The constants are pessimistic, because the weight floor, about 0.03, enters cubed. The rules decrease with activity, so for data bounded by R the squared sigmoid floor improves to ten to the minus three plus 0.03 over one plus e to the R. Pixels are independent, so the bound holds pixel by pixel.

Chapter 4 is different. Its observation weights are recomputed from the current profiles, so its estimator is a fixed point and needs the contraction condition in the stereo backups. Chapter 5 lists this as a future direction. The bound covers the step from the represented data to the estimate; connecting it to raw event perturbations is the representation question on slide 29.

BACKUP 66 / DECK 96

How the parameters were chosen

Likely question: How did you choose the regularization parameters, and why not a standard rule such as the discrepancy principle?

[Read the table one row at a time, then point to the two blocks.]

Answer

Each was chosen by grid sweep. In Chapter 2 we swept the scale of each adaptive rule, and separately the kernel type and width, then chose the kernel and rule together as the configuration with the best average rank over the nine evaluation metrics. Antil and Sayre use one scalar magnitude weight on the image; making that weight activity-adaptive, and comparing three ways to do it, is the change Chapter 2 evaluates.

For flow, the smoothness weight came from grid sweeps on MVSEC, for each solver. In stereo, the coupling weight and the number of latent images came from a grid sweep, and inverse-variance weighting from a paired comparison, on the four evaluated sequences.

The theorems hold for every positive value of these parameters. So the choice moves bias and accuracy, and the solve stays unique and stable whatever value the sweep picks.

If asked for more

The discrepancy principle chooses the strongest regularization whose residual matches the noise level, so it needs that level. For raw signed counts, the Skellam model gives the variance as the sum of the ON and OFF rates, which can be estimated from the counts; after the Chapter 4 transformation, that variance has to be propagated through it. The L-curve and generalized cross-validation need no noise level, so they apply directly.

The sweeps used the same sequences we report, so a held-out protocol is the clean next experiment. The data-driven route is bilevel learning of the weights, from the research-opportunities slide.

BACKUP 67 / DECK 97

Learned estimators face the same free directions

Likely question: Why not train a neural network to do this?

[Read the "left free by the events" column first, then the two choice columns.]

Answer

A trained network is also a regularized estimator. Where the events leave the answer free, it fills the gap from its training images; my methods fill it with a penalty that is written down and analyzed.

The dissertation identifies those free directions. In the temporal problem it's the constant brightness offset at each pixel. In flow it's the motion component along a straight edge. In stereo it's the disparity candidates whose temporal profiles coincide. Every estimator supplies those from a model or from training data. That's also why event reconstruction has no agreed standard yet: scoring against frame images assumes a brightness level the events leave open.

The two approaches combine naturally. The weights can be learned in a bilevel problem, and because each fixed-weight solve is unique and smooth in the weights, the outer problem has a well-defined, smooth lower level. And for data like the ISS and ballistics sequences, the reconstruction runs from events alone, with no reference images or training set.

If asked for more

If the question is benchmark accuracy: on the public flow and stereo benchmarks, learned methods report the lowest errors. The dissertation asks a different question, what the data determine and when each estimator is stable, and its stereo comparison is against model-based event methods.

A second combination is unrolling: run a fixed number of block-Jacobi or IRLS iterations and treat each one as a network layer. The convergence results then describe what each layer does.

If asked whether the bilevel problem is well posed

The lower level is. If the weights range over a box between a positive floor and a ceiling, and the outer loss is continuous, the reduced outer objective is continuous on a compact set, so an outer minimizer exists. Uniqueness of that minimizer, generalization of the learned weights, and convergence of a particular outer algorithm each need their own analysis.

BACKUP 68 / DECK 98

The spline regularizer is uniformly coercive under refinement

Likely question: Your ridge, two epsilon over M, goes to zero as you refine the knots. Doesn't coercivity degrade?

[Point to step 1, then to the two terms of step 2.]

Answer

Only in coefficient space. The cubic B-splines are nonnegative and sum to one, so at every point the squared spline value is at most a weighted average of the squared coefficients, with the basis functions as weights. Integrating, and using that each basis function integrates to at most h squared, gives the L two norm of the spline at most h squared times the squared coefficient norm.

So the ridge, two epsilon over M times the squared coefficients, controls two epsilon over M h squared times the squared L two norm, and M h squared tends to the area of the domain; on our frame at h equal to eight pixels the ratio is 1.20. The eigenvalue floor in the diffusion tensor controls the gradient. Together, the discrete regularizer controls the full H one norm with a constant that holds for every knot spacing.

What does grow is the condition number of the coefficient matrix, like h to the minus two. That's the usual growth for an H one Galerkin matrix with a mass term.

If asked for more

The largest eigenvalue of the stiffness matrix stays bounded in two dimensions: its entries are of the size of the diffusion tensor, and each basis function overlaps at most 49 others, so Gershgorin bounds it. The smallest eigenvalue is the ridge, of order h squared. Knot-spacing continuation, which the implementation supports, is the usual remedy for iteration counts. I checked the mass bound numerically with the clamped cubic basis: the largest one-dimensional Gram eigenvalue divided by h is 0.981, 0.994, and 1.000 at spacings of 8, 4, and 2 pixels. With nonuniform knots, weighting each coefficient by its basis integral makes the ridge a lumped mass term again.

BACKUP 69 / DECK 99

Weak regularization: a limit in time, the aperture line in space

Likely question: What happens to your results as the regularization goes to zero?

[Left column first, then the right column.]

Answer

The two problems behave differently, and the difference shows what each penalty contributes.

In time, the solution has a clean limit. Testing the normal equations with the constant vector removes the data term at every scale, so the weighted zero-mean rule that picks the baseline is the same for every lambda. As lambda goes to zero, the solution converges to the exact fit of the increments with the smallest weighted norm; Chapter 2 states that limit. What the penalty buys is noise control. With a scalar penalty, the limit is the running sum of the increments, centered. Noise in the increments accumulates along the chain, with gain one over root g one, about N t over pi. The penalty caps that gain at one over two lambda.

In space, the smoothing supplies identifiability. An event-free region gives a test field whose energy is only alpha squared times its gradient, so the coercivity constant is at most alpha squared. At alpha equal to zero the energy is pointwise, and for the time surface its minimizers at each pixel are the whole aperture line from slide 23.

If asked for more

The full-space condition number, g max plus lambda squared over lambda squared, grows only along the constant mode, which the data never excite; Backup 17 removes that mode. Convergence to the limit sets in once lambda squared is small compared with g one, roughly pi squared over N t squared. For the structure tensor at alpha zero, pixels with a rank-two pooled block keep a unique velocity; the rest keep a line or the whole plane.

BACKUP 70 / DECK 100

Where there are no events, smoothness fills in harmonically

Likely question: In regions with no events, isn't the flow simply invented?

[Point to the weak equation, then to the maximum-principle bound.]

Answer

The regularizer chooses those values, and we can say exactly how. Take a connected region U with no represented gradient, so the data matrix and the forcing vanish there. Testing the weak equation with fields supported in U leaves only the gradient term, so each velocity component is harmonic in U. Equivalently, among fields that agree with the estimate on U's boundary, it has the least gradient energy.

That gives a maximum principle. Pick any direction e, and suppose the velocity component along e is at most M on Gamma, the interface with the event-active pixels. Test with the positive part of that component minus M. The weak equation says its gradient energy is zero, and it vanishes on Gamma, so it's zero throughout U. Doing this for every direction bounds the speed inside U by the largest speed on its boundary.

So an event-free region gets the smoothest interpolation of the motion measured around it, with no new extremes. The theorem guarantees a unique regularized estimate; identifying the physical motion at each pixel needs data there.

If asked for more

Where U reaches the image border, the natural Neumann condition holds and adds no constraint, because test fields are free on the outer boundary. All three representations have residual-square data terms, so the forcing vanishes wherever the data matrix does. On the pixel graph, the same statement says an event-free node's velocity is the weighted average of its neighbors'. A motion boundary inside an event-free region is smoothed across; that's the H one modeling choice from Backup 27.

BACKUP 71 / DECK 101

A point measure needs a representation before optimization

Likely question: Why is representing the event stream part of the model?

Answer

We can write the raw event stream as a signed point measure: a point mass at each event's location and time, weighted by its polarity.

[Follow the three boxes.]

The quadratic models in this talk need quantities that can go into a residual, so first we convert the stream. Binning, smoothing, and interpolation do that.

For temporal reconstruction, kernels spread each event's contribution across reconstruction intervals. For flow, time surfaces and voxel grids supply the coefficients of the motion constraints. For stereo, accumulated events give paired temporal features.

This conversion is part of the model. It decides what information we keep, and how measurement errors reach the coefficients.

I'll also keep track of what each variable means. Physical intensity, log intensity, represented event activity, and a fitted latent profile all play different roles. For example, the stereo preprocessing produces event-feature profiles whose scale is set by that preprocessing.

[If asked about the continuous-time meaning of the event measure: Backup 03, page 33. If asked why the kernel choice matters: Backup 05, page 35.]

BACKUP 72 / DECK 102

Derive the time-surface constraint by following the edge

Likely question: Where does the time-surface constraint come from?

Answer

Define psi as the current time minus the crossing-time map. Follow a point moving with the edge: along that path, psi stays zero, because the current time equals the crossing time there.

Differentiate along the path. The time derivative gives one. The spatial part gives minus the gradient of the crossing-time map, dotted with the velocity. Setting the sum to zero gives the time-surface constraint: the gradient of the crossing-time map, dotted with the velocity, equals one.

In scene terms, the size of that gradient is the edge's slowness. If an edge takes ten milliseconds to cross one pixel, it's moving at one hundred pixels per second perpendicular to itself. Motion along the edge leaves the crossing times unchanged, so this single equation already contains the aperture problem.

[Pause at the hypotheses box.]

The derivation assumes each location is crossed once, the normal speed is nonzero, and the crossing-time map is differentiable. In real data, a stored timestamp can be stale, or a location can be crossed more than once. Fitting and interpolation also affect the coefficients.

So we treat this relation as a model for the represented data. The analysis then shows what follows once the coefficient fields meet the stated assumptions.

[If asked how the time-surface gradient is estimated and how accurate it is: Backup 42, page 72; for the neighborhood size, Backup 15, page 45. If asked about correlated timestamp errors: Backup 40, page 70.]

BACKUP 73 / DECK 103

Voxel projection: transport a represented event-activity field

Likely question: What does voxel projection assume?

Answer

A voxel grid adds a time axis to the image grid. After smoothing and binning, it gives an event-activity field, V.

Integrate V over a sliding time window to get a.

[Point to the assumption box.]

This step adds a modeling assumption, shown in the box: the activity field moves with the scene, at constant velocity over the window. That goes beyond the threshold model. A pixel can start or stop producing events as brightness, motion, and threshold conditions change, and activity is a different quantity from brightness: a steadily bright region can produce very little of it.

Integrating the transport equation gives another transport equation, now for a. Its time derivative is the activity entering at the end of the window minus the activity leaving at the start.

The residual is then the spatial gradient of a, dotted with the velocity, plus that time derivative.

The continuous identity holds under the transport assumption with consistent derivatives. Binning and the choice of numerical derivatives can introduce mismatch.

[If asked why event activity should obey transport, or why not a continuity equation: Backup 39, page 69. If asked whether the temporal slope and spatial projection must match: Backup 16, page 46.]

BACKUP 74 / DECK 104

The structure tensor pools directional information

Likely question: How does the structure tensor constrain both velocity components?

Answer

The structure tensor pools several nearby gradients of the represented event field.

Each space-time gradient contributes an outer product. With nonnegative pooling weights, the result is positive semidefinite, because its quadratic form is an integral of squares. Contracting J with the augmented velocity gives the pooled squared transport residual: for one gradient, the residual is V_x v_x plus V_y v_y plus V_t, and its square is the augmented velocity times that gradient's outer product times the augmented velocity.

[Point to the spatial block and the temporal cross term.]

The spatial block gives the quadratic dependence on the two velocity components. The temporal cross terms give the linear term. The last temporal entry adds a constant to the energy, because the third component of the augmented velocity is fixed at one.

A single spatial gradient constrains one direction. Pooling gradients that point in different directions can constrain both components at one location.

That's one way to get more directional information. The flow theorem requires edge directions to be covered somewhere in the domain, and spatial smoothness carries that information to every pixel.

The difference from voxel projection is where the averaging happens. Voxel projection averages over time, assuming constant velocity across the time window, and each pixel contributes a single constraint. The structure tensor averages over space, assuming constant velocity across a small neighborhood. The neighborhood size sets the trade-off between identifying both components and resolving motion boundaries.

[If asked when sampled gradient directions suffice: Backup 11, page 41. If asked whether integrated directional information is necessary: Backup 35, page 65.]

BACKUP 75 / DECK 105

Proof idea: smoothness fixes variation, edges fix the drift

Likely question: How does the proof of the flow theorem go?

Answer

[Point to the decomposition, then move across the three columns.]

Write the velocity as its spatial mean plus a zero-mean fluctuation, w. Physically, the mean is the common drift of the whole field, like a camera pan, and the fluctuation is how motion varies across the image: rotation, zoom, or objects moving separately.

Poincaré's inequality bounds the fluctuation by the gradient, so the smoothness term controls it.

Smoothness charges nothing for a uniform pan, so the data have to fix the mean. That's the role of m, the smallest eigenvalue of the integrated matrix B. It measures how well the scene's edges, taken together, cover every direction of motion. Because m is positive, the data energy controls the mean.

Together, the energy controls the full H one norm, with a constant c zero that depends on the domain, alpha, the bound on the data matrix, and m. Lax–Milgram then gives the unique minimizer.

The constant c zero shrinks as m shrinks: nearly parallel edges weaken control of the drift. The stripe scene of slide 26 comes close to that case.

[If asked for the derivations: the full proof is Backup 09, page 39. If asked about stability: the same c zero gives a Lipschitz bound in B and b, Backup 10, page 40. If asked what happens when the integrated matrix is singular: a constant velocity direction is uncontrolled, Backup 35. If asked whether the flow is stable with respect to the raw events: Backup 36, page 66. If asked whether one mechanism runs through all three uniqueness results: Backup 64, page 94.]

BACKUP 76 / DECK 106

Stereo on MVSEC: competitive accuracy, more scored points

Likely question: How does the stereo method compare with all the methods on MVSEC?

Answer

[Point to the energy first, then to the WLS row and the Piatkowska row.]

At each candidate disparity, the model solves for one temporal profile per view. Each profile fits its own events, as in the temporal chapter, and a soft penalty ties the two profiles together.

[Point to the two labels.]

The mathematical result is for a fixed disparity with the weights frozen. The coupling term is nonnegative, so the temporal coercivity proof carries over, and the inner problem has a unique solution. The benchmark runs the full method: weight updates, cost aggregation, and candidate selection.

On MVSEC, here's how that performs on the indoor flying sequences, against the dataset's lidar-based ground-truth depth. The first four columns are mean depth error in centimeters, one per sequence. Then log RMSE, and the number of scored points.

Our weighted least-squares method, WLS, has the lowest error on indoor flying two and the lowest log RMSE. On the other three sequences, the local adaptation of Piatkowska and colleagues' method is lower, by between one and a half and six centimeters. WLS scores two point six six million points, more than twice the one point two million for that method. It also leads the three other prior event-based methods on every sequence.

Each method is scored on its own valid pixels, so the comparison reflects both accuracy and coverage.

[If asked why the inner solve is unique: Backup 51, page 81. If asked whether a unique solve guarantees the right disparity: Backup 54, page 84; the conditions that do are Backup 55, page 85, and Backup 56, page 86; for noise tolerance, Backup 19, page 49. If asked whether IRLS converges: Backup 59, page 89. If asked whether it is maximum likelihood or majorization–minimization: Backup 21, page 51, and Backup 28, page 58. If asked whether the comparison is fair: Backup 60, page 90, and Backup 23, page 53. If asked what supports the reciprocal weights: Backup 61, page 91. If asked what increasing the coupling does: Backup 52, page 82. If asked about hard fusion: Backup 62, page 92. For exactly which competitors beat the ideal match: Backup 18, page 48. For the IRLS contraction constant and what one IRLS call returns: Backup 20, page 50, and Backup 22, page 52. If asked about the dataset's own ground-truth limits: Backup 60, page 90.]

BACKUP 77 / DECK 107

From event changes to well-posed estimators

[Closing summary; use it if the discussion returns to the common question.]

I began by asking what we can recover from changes alone.

Temporal reconstruction shows how a penalty selects the baseline. Flow shows how directional observations and spatial coupling together determine a motion field. Stereo shows that a unique inner solution and a reliable match need separate arguments.

Across all three, the common question is which directions the observations control, and which the model chooses.

[For questions: the discussion guide is Backup 01, page 31; flow detail starts at Backup 41, page 71; the stereo arc at Backup 47, page 77; references at page 110.]

BACKUP 78 / DECK 108

How big is the temporal monocular problem?

Likely question: How many unknowns are there, what does the solve cost, and what would you change for low-power use?

Answer

There are H times W times N_t unknowns, split into independent temporal histories. Each pixel has a positive-definite tridiagonal system. Weights, right-hand side, and solve each take O(N_t), giving O(H W N_t) total work.

With E events and at most k bins per event, direct indexing on a uniform grid gives O(E k + H W N_t) including accumulation, for fixed kernel order. Delta binning has k equal to one. The edge solver uses this structure; the main Python path uses cubic dense solves.

A 346 by 260 sensor with fifty samples has about four and a half million unknowns: eighteen megabytes of float32 output alone. Newest-endpoint streaming takes O(H W) state and work per interval. It matches the growing-prefix batch endpoint for finalized inputs; rolling windows change the boundary problem.

If asked for more

Preserve the estimator. Bin directly, reuse buffers, and skip entirely zero observation histories. Zero net sum can hide changes. Reuse factors across identical weight profiles; cache weights for repeated integer counts. Quiet pixels with earlier activity still need state updates.

Trade resources deliberately. ROI reduces coverage; coarser sampling reduces detail; shorter windows change history. Symmetric kernels add look-ahead latency. Float32, quantized weights, and fixed-point arithmetic require error checks, especially at weak regularization. Tune thread count for joules per estimate under the latency deadline. Measure peak memory too.

For detection, event counts or a leaky activity state may suffice. Asynchronous high-pass or complementary filters offer another compact estimator, including an events-only option. Their objective and baseline behavior differ from this model: benchmark task accuracy alongside cost.

Sources: Chapter 2 normal equations; edge implementation and README; Scheerlinck et al., Continuous-time Intensity Estimation Using Event Cameras (ACCV 2018). CPU timings: Backup 07, page 37.

BACKUP 79 / DECK 109

One CPU core streams about 114 million pixel updates per second, with 16 bytes of state per pixel

Likely question: How fast is it on one core, and what would fit on a small, low-power board?

Answer

Because each streaming step does constant work per pixel, the cost comes down to one rate. On one CPU core, the closest stand-in for a small board, the method performs about 114 million pixel updates per second, just under nine nanoseconds each, and keeps sixteen bytes of state per pixel.

The two dots are two recordings, each producing a new frame every two milliseconds. They differ in resolution and by a factor of nine in event rate, and they give the same rate to within a third of a percent. The labels are processing time divided by recording time, so anything below one keeps up. The 346 by 260 recording streams at 0.39 of real time; the 640 by 480 recording streams at 1.3 on one core and 0.21 on eight.

So for any sensor, the budget is pixels times frames per second, below 114 million per core. At 500 frames per second that's about 229 thousand pixels per core; at 100 frames per second, about 1.1 million.

Batch reconstruction also follows the scene: in a typical window only about seven percent of pixels have events, and skipping the rest cuts the work tenfold.

[For multithread timings: Backup 07, page 37. For problem size, memory, and low-power strategies: Backup 78, page 108.]

BACKUP 80 / DECK 110

References (1 of 3)

[Navigation only. Silent navigation cue: the reference numbers match the dissertation bibliography; each bracketed number in the deck links here. Return with the viewer's back command.]

BACKUP 81 / DECK 111

References (2 of 3)

[Navigation only. Silent navigation cue: the reference numbers match the dissertation bibliography; each bracketed number in the deck links here. Return with the viewer's back command.]

BACKUP 82 / DECK 112

References (3 of 3)

[Navigation only. Silent navigation cue: the reference numbers match the dissertation bibliography; each bracketed number in the deck links here. Return with the viewer's back command.]